```
# ==========================================
# PHASE 3 - DATASET INITIALIZATION
# ==========================================

import kagglehub
import pandas as pd
import os
import numpy as np

print("Downloading / locating Food.com dataset...")

path = kagglehub.dataset_download(
    "shuyangli94/food-com-recipes-and-user-interactions"
)

# Load original CSV files
recipes_raw = pd.read_csv(
    os.path.join(path, "RAW_recipes.csv")
)

interactions_raw = pd.read_csv(
    os.path.join(path, "RAW_interactions.csv")
)

# Create working copies
recipes = recipes_raw.copy(deep=True)
interactions = interactions_raw.copy(deep=True)

print("\nDATASETS LOADED SUCCESSFULLY!")

print("Recipes shape:", recipes.shape)
print("Interactions shape:", interactions.shape)

display(recipes.head(3))
display(interactions.head(3))
```

```
Downloading / locating Food.com dataset...
Using Colab cache for faster access to the 'food-com-recipes-and-user-interactions' dataset.

DATASETS LOADED SUCCESSFULLY!
Recipes shape: (231637, 12)
Interactions shape: (1132367, 5)
```

	name	id	minutes	contributor_id	submitted	tags	nutrition	n_steps	steps	description	ingredients	
0	arriba baked winter squash mexican style	137739	55	47892	2005-09-16	['60-minutes-or-less', 'time-to-make', 'course...	[51.5, 0.0, 13.0, 0.0, 2.0, 0.0, 4.0]	11	['make a choice and proceed with recipe', 'dep...	autumn is my favorite time of year to cook! th...	['winter squash', 'mexican seasoning', 'mixed ...	
1	a bit different breakfast pizza	31490	30	26278	2002-06-17	['30-minutes-or-less', 'time-to-make', 'course...	[173.4, 18.0, 0.0, 17.0, 22.0, 35.0, 1.0]	9	['preheat oven to 425 degrees f', 'press dough...	this recipe calls for the crust to be prebaked...	['prepared pizza crust', 'sausage patty', 'egg...	
2	all in the kitchen chili	112140	130	196586	2005-02-25	['time-to-make', 'course', 'preparation', 'mai...	[269.8, 22.0, 32.0, 48.0, 39.0, 27.0, 5.0]	6	['brown ground beef in large pot', 'add choppe...	this modified version of 'mom's' chili was a h...	['ground beef', 'yellow onions', 'diced tomato...	

	user_id	recipe_id	date	rating	review
0	38094	40893	2003-02-17	4	Great with a salad. Cooked on top of stove for...
1	1293707	40893	2011-12-21	5	So simple, so delicious! Great for chilly fall...
2	8937	44394	2002-12-01	4	This worked very well and is EASY. I used not...

```
# ==============================================================
# PHASE 3: DATA PREPROCESSING AND FEATURE ENGINEERING
# Food.com Personalized Nutrition-Aware Recommendation System
# ==============================================================
```

```
import pandas as pd
import numpy as np
import ast
import re
import os
import zipfile

from collections import Counter
from IPython.display import display

print("=" * 75)
print("PHASE 3: DATA PREPROCESSING AND FEATURE ENGINEERING")
print("=" * 75)

# ============================================================
# 1. PRESERVE AND LOAD ORIGINAL DATASETS
# ============================================================

# Use original datasets to avoid repeated preprocessing
if "recipes_raw" not in globals():
    recipes_raw = recipes.copy(deep=True)

if "interactions_raw" not in globals():
    interactions_raw = interactions.copy(deep=True)

recipes = recipes_raw.copy(deep=True)
interactions = interactions_raw.copy(deep=True)

print("\n1. ORIGINAL DATASETS")
print("Recipes:", recipes.shape)
print("Interactions:", interactions.shape)


# ============================================================
# 2. DEFINE HELPER FUNCTIONS
# ============================================================

def parse_list(value):
    """Safely convert string representations of lists into lists."""

    try:
        if isinstance(value, list):
            return value

        if pd.isna(value):
            return []

        result = ast.literal_eval(value)

        if isinstance(result, list):
            return result

        return []

    except (ValueError, SyntaxError, TypeError):
        return []


def clean_text(value):
    """Normalize text by lowercasing and removing extra spaces."""

    value = str(value).lower().strip()
    value = re.sub(r"\s+", " ", value)

    return value


# ============================================================
# 3. PREPARATION TIME CLEANING
# ============================================================

recipes["minutes"] = pd.to_numeric(
    recipes["minutes"],
    errors="coerce"
)

recipes["minutes_clean"] = recipes["minutes"].copy()

# Zero and negative preparation times are treated as unavailable
recipes.loc[
    recipes["minutes_clean"] <= 0,
    "minutes_clean"
```

```
] = np.nan

# Remove the known invalid integer-overflow value
recipes.loc[
    recipes["minutes_clean"] == 2147483647,
    "minutes_clean"
] = np.nan

# Remove infinite values if present
recipes["minutes_clean"] = recipes["minutes_clean"].replace(
    [np.inf, -np.inf],
    np.nan
)


def categorize_time(minutes):

    if pd.isna(minutes):
        return "Unknown"

    elif minutes <= 15:
        return "Quick"

    elif minutes <= 30:
        return "Short"

    elif minutes <= 60:
        return "Medium"

    else:
        return "Long"


recipes["time_category"] = recipes["minutes_clean"].apply(
    categorize_time
)

print("\n2. PREPARATION TIME")
print("Missing/invalid preparation times:",
      recipes["minutes_clean"].isna().sum())

print(recipes["time_category"].value_counts())


# ================================================================
# 4. NUTRITION PARSING
# ================================================================

nutrition_columns = [
    "calories",
    "total_fat_pdv",
    "sugar_pdv",
    "sodium_pdv",
    "protein_pdv",
    "saturated_fat_pdv",
    "carbohydrates_pdv"
]

def parse_nutrition(value):

    parsed = parse_list(value)

    if len(parsed) != 7:
        return [np.nan] * 7

    try:
        return [float(x) for x in parsed]

    except (ValueError, TypeError):
        return [np.nan] * 7


nutrition_values = recipes["nutrition"].apply(
    parse_nutrition
)

nutrition_df = pd.DataFrame(
    nutrition_values.tolist(),
    columns=nutrition_columns,
    index=recipes.index
)
```

```
recipes[nutrition_columns] = nutrition_df

# Convert invalid infinite values into missing values
recipes[nutrition_columns] = recipes[
    nutrition_columns
].replace([np.inf, -np.inf], np.nan)

print("\n3. NUTRITION PARSING")
print("Nutrition columns created:", len(nutrition_columns))
print("Missing nutritional values:")
print(recipes[nutrition_columns].isna().sum())


# ============================================================
# 5. NUTRITIONAL FEATURE ENGINEERING
# ============================================================

# Preserve original nutrition values.
# Create outlier indicators and log-transformed features.

for column in nutrition_columns:

    valid_values = recipes[column].dropna()

    if len(valid_values) > 0:

        threshold = valid_values.quantile(0.999)

        recipes[column + "_outlier"] = (
            recipes[column] > threshold
        ).fillna(False)

    else:

        recipes[column + "_outlier"] = False

    # Log transformation for numerical modelling
    recipes[column + "_log"] = np.log1p(
        recipes[column].clip(lower=0)
    )

print("\n4. NUTRITIONAL FEATURE ENGINEERING COMPLETED")

outlier_summary = pd.DataFrame({
    "Feature": nutrition_columns,
    "Outlier_Count": [
        recipes[column + "_outlier"].sum()
        for column in nutrition_columns
    ]
})

display(outlier_summary)


# ============================================================
# 6. INGREDIENT PROCESSING
# ============================================================

recipes["ingredients_list"] = recipes["ingredients"].apply(
    parse_list
)

recipes["ingredients_clean"] = recipes[
    "ingredients_list"
].apply(
    lambda items: [
        clean_text(item)
        for item in items
        if str(item).strip()
    ]
)

# Convert ingredient list into text for TF-IDF later
recipes["ingredients_text"] = recipes[
    "ingredients_clean"
].apply(
    lambda items: " ".join(items)
)

# Count ingredients after parsing
recipes["ingredient_count_clean"] = recipes[
    "ingredients_clean"
```

```
].apply(len)

# Calculate most common ingredients
ingredient_counter = Counter()

for ingredient_list in recipes["ingredients_clean"]:

    ingredient_counter.update(set(ingredient_list))

ingredient_frequency = pd.DataFrame(
    ingredient_counter.most_common(20),
    columns=["Ingredient", "Recipe_Count"]
)

print("\n5. INGREDIENT PROCESSING COMPLETED")

print("Recipes with empty ingredient lists:",
      recipes["ingredients_clean"].apply(len).eq(0).sum())

print("\nTOP 20 INGREDIENTS")
display(ingredient_frequency)


# ================================================================
# 7. TAG PROCESSING
# ================================================================

recipes["tags_list"] = recipes["tags"].apply(
    parse_list
)

recipes["tags_clean"] = recipes["tags_list"].apply(
    lambda items: [
        clean_text(tag)
        for tag in items
        if str(tag).strip()
    ]
)

# Convert tags into text for later recommendation modelling
recipes["tags_text"] = recipes["tags_clean"].apply(
    lambda items: " ".join(items)
)

# Calculate most common tags
tag_counter = Counter()

for tag_list in recipes["tags_clean"]:

    tag_counter.update(set(tag_list))

tag_frequency = pd.DataFrame(
    tag_counter.most_common(20),
    columns=["Tag", "Recipe_Count"]
)

print("\n6. TAG PROCESSING COMPLETED")

print("Recipes with empty tag lists:",
      recipes["tags_clean"].apply(len).eq(0).sum())

print("\nTOP 20 TAGS")
display(tag_frequency)


# ================================================================
# 8. MISSING VALUE HANDLING
# ================================================================

# Replace missing names with a placeholder
recipes["name"] = recipes["name"].fillna(
    "Untitled recipe"
)

# Missing descriptions are retained as empty text
recipes["description"] = recipes["description"].fillna(
    ""
)

# Create a cleaned description feature
recipes["description_clean"] = recipes[
    "description"
```

```
].apply(clean_text)

print("\n7. MISSING VALUE HANDLING")

print(
    recipes[
        ["name", "description", "description_clean"]
    ].isna().sum()
)


# ================================================================
# 9. INTERACTION DATA PREPROCESSING
# ================================================================

# Preserve all interactions
interactions_all = interactions.copy(deep=True)

# Convert dates
interactions_all["date"] = pd.to_datetime(
    interactions_all["date"],
    errors="coerce"
)

# Convert ratings into numeric values
interactions_all["rating"] = pd.to_numeric(
    interactions_all["rating"],
    errors="coerce"
)

# Explicit ratings: 1 to 5
interactions_rated = interactions_all[
    interactions_all["rating"].between(1, 5)
].copy()

# Zero-rated interactions are kept separately
interactions_zero = interactions_all[
    interactions_all["rating"] == 0
].copy()

print("\n8. INTERACTION PREPROCESSING")

print("All interactions:", len(interactions_all))
print("Explicitly rated interactions:", len(interactions_rated))
print("Zero-rated interactions:", len(interactions_zero))


# ================================================================
# 10. CHECK REPEATED USER-RECIPE INTERACTIONS
# ================================================================

duplicate_mask = interactions_rated.duplicated(
    subset=["user_id", "recipe_id"],
    keep=False
)

repeated_interaction_rows = duplicate_mask.sum()

repeated_interaction_pairs = (
    interactions_rated.loc[
        duplicate_mask,
        ["user_id", "recipe_id"]
    ].drop_duplicates().shape[0]
)

print("\n9. REPEATED USER-RECIPE INTERACTIONS")

print("Rows belonging to repeated pairs:",
      repeated_interaction_rows)

print("Number of repeated user-recipe pairs:",
      repeated_interaction_pairs)


# ================================================================
# 11. AGGREGATE EXPLICIT RATINGS
# ================================================================

# One record per user-recipe pair.
# Average ratings are retained, along with the number of ratings.

interactions_processed = (
```

```
    interactions_rated
    .groupby(
        ["user_id", "recipe_id"],
        as_index=False
    )
    .agg(
        rating=("rating", "mean"),
        rating_count=("rating", "count")
    )
)

print("\n10. PROCESSED INTERACTION DATA")
print("Shape:", interactions_processed.shape)

display(interactions_processed.head())


# ============================================================
# 12. VALIDATE RECIPE IDS
# ============================================================

recipe_ids = set(recipes["id"].dropna())

interaction_recipe_ids = set(
    interactions_all["recipe_id"].dropna()
)

unmatched_recipe_ids = (
    interaction_recipe_ids - recipe_ids
)

print("\n11. RECIPE ID VALIDATION")

print("Unique recipe IDs:", len(recipe_ids))

print("Unique recipe IDs in interactions:",
      len(interaction_recipe_ids))

print("Unmatched interaction recipe IDs:",
      len(unmatched_recipe_ids))


# ============================================================
# 13. CREATE FINAL PROCESSED DATASETS
# ============================================================

recipes_processed = recipes.copy(deep=True)

# Do not remove recipes unnecessarily.
# Original recipe IDs were already verified as unique.

print("\n12. FINAL DATASET DIMENSIONS")

print("Recipes processed:", recipes_processed.shape)
print("Interactions processed:", interactions_processed.shape)
print("All interactions:", interactions_all.shape)
print("Zero-rated interactions:", interactions_zero.shape)


# ============================================================
# 14. FINAL DATA QUALITY VALIDATION
# ============================================================

print("\n" + "=" * 75)
print("PHASE 3: FINAL DATA QUALITY REPORT")
print("=" * 75)

print("\nRECIPE DATASET")

print("Total recipes:", len(recipes_processed))

print("Unique recipe IDs:",
      recipes_processed["id"].nunique())

print("Duplicate recipe IDs:",
      recipes_processed["id"].duplicated().sum())

print("\nMISSING VALUES")

print(
    recipes_processed[
        [
```

```
            "name",
            "description",
            "ingredients_text",
            "tags_text",
            "minutes_clean"
        ] + nutrition_columns
    ].isna().sum()
)

print("\nPREPARATION TIME CATEGORIES")

print(recipes_processed["time_category"].value_counts())

print("\nINGREDIENT VALIDATION")

print(
    "Empty ingredient lists:",
    recipes_processed["ingredients_clean"].apply(len).eq(0).sum()
)

print("\nTAG VALIDATION")

print(
    "Empty tag lists:",
    recipes_processed["tags_clean"].apply(len).eq(0).sum()
)

print("\nNUTRITION VALIDATION")

print("Nutrition columns:", len(nutrition_columns))

print("Missing nutrition values:",
      recipes_processed[nutrition_columns].isna().sum().sum())

print("\nINTERACTION VALIDATION")

print("All interactions:", len(interactions_all))

print("Explicit ratings:", len(interactions_rated))

print("Zero ratings:", len(interactions_zero))

print("Aggregated user-recipe pairs:",
      len(interactions_processed))

print("Unmatched recipe IDs:",
      len(unmatched_recipe_ids))


# ================================================================
# 15. EXPORT PROCESSED DATASETS
# ================================================================

output_dir = "/content/phase3_output"

os.makedirs(output_dir, exist_ok=True)

recipes_processed.to_csv(
    f"{output_dir}/recipes_processed.csv",
    index=False
)

interactions_processed.to_csv(
    f"{output_dir}/interactions_processed.csv",
    index=False
)

interactions_all.to_csv(
    f"{output_dir}/interactions_all.csv",
    index=False
)

interactions_zero.to_csv(
    f"{output_dir}/interactions_zero.csv",
    index=False
)

ingredient_frequency.to_csv(
    f"{output_dir}/top_ingredients.csv",
    index=False
)
```

```
tag_frequency.to_csv(
    f"{output_dir}/top_tags.csv",
    index=False
)

outlier_summary.to_csv(
    f"{output_dir}/nutrition_outlier_summary.csv",
    index=False
)

# Package all outputs into one ZIP file
zip_path = "/content/Phase3_Processed_Datasets.zip"

with zipfile.ZipFile(
    zip_path,
    "w",
    zipfile.ZIP_DEFLATED
) as zip_file:

    for filename in os.listdir(output_dir):

        filepath = os.path.join(output_dir, filename)

        zip_file.write(
            filepath,
            arcname=filename
        )


# ================================================================
# 16. FINAL SUMMARY
# ================================================================

print("\n" + "=" * 75)
print("PHASE 3 PROCESSING COMPLETED")
print("=" * 75)

print("\nFiles created:")

for filename in os.listdir(output_dir):
    print("-", filename)

print("\nZIP FILE:", zip_path)

print("\nYour processed datasets are ready for Phase 4!")

# Automatically download all processed datasets in one ZIP
from google.colab import files

files.download(zip_path)
```

```
================================================================================
PHASE 3: DATA PREPROCESSING AND FEATURE ENGINEERING
================================================================================

1. ORIGINAL DATASETS
Recipes: (231637, 12)
Interactions: (1132367, 5)

2. PREPARATION TIME
Missing/invalid preparation times: 1095
time_category
Medium     70077
Long       62506
Short      55131
Quick      42828
Unknown     1095
Name: count, dtype: int64

3. NUTRITION PARSING
Nutrition columns created: 7
Missing nutritional values:
calories             0
total_fat_pdv        0
sugar_pdv            0
sodium_pdv           0
protein_pdv          0
saturated_fat_pdv    0
carbohydrates_pdv    0
dtype: int64

4. NUTRITIONAL FEATURE ENGINEERING COMPLETED
```

	Feature	Outlier_Count
0	calories	232
1	total_fat_pdv	232
2	sugar_pdv	232
3	sodium_pdv	230
4	protein_pdv	231
5	saturated_fat_pdv	226
6	carbohydrates_pdv	230

```
5. INGREDIENT PROCESSING COMPLETED
Recipes with empty ingredient lists: 0

TOP 20 INGREDIENTS
```

	Ingredient	Recipe_Count
0	salt	85746
1	butter	54975
2	sugar	44535
3	onion	39065
4	water	34914
5	eggs	33761
6	olive oil	32822
7	flour	26266
8	milk	25786
9	garlic cloves	25748
10	pepper	22319
11	brown sugar	18655
12	garlic	18087
13	all-purpose flour	17659
14	baking powder	17504
15	egg	17304
16	salt and pepper	15415
17	parmesan cheese	14807
18	lemon juice	14233
19	baking soda	14099

```
6. TAG PROCESSING COMPLETED
```

```
Recipes with empty tag lists: 109

TOP 20 TAGS
```

	Tag	Recipe_Count
0	preparation	230546
1	time-to-make	225326
2	course	218148
3	main-ingredient	170446
4	dietary	165091
5	easy	126062
6	occasion	114145
7	cuisine	91165
8	low-in-something	85776
9	main-dish	71786
10	equipment	70436
11	60-minutes-or-less	69990
12	number-of-servings	58949
13	meat	56042
14	30-minutes-or-less	55077
15	vegetables	53814
16	taste-mood	52143
17	4-hours-or-less	49497
18	north-american	48479
19	3-steps-or-less	44933

```
7. MISSING VALUE HANDLING
name                0
description         0
description_clean   0
dtype: int64

8. INTERACTION PREPROCESSING
All interactions: 1132367
Explicitly rated interactions: 1071520
Zero-rated interactions: 60847

9. REPEATED USER-RECIPE INTERACTIONS
Rows belonging to repeated pairs: 0
Number of repeated user-recipe pairs: 0

10. PROCESSED INTERACTION DATA
Shape: (1071520, 4)
```

Next steps: Generate code with `rating_summary` | New interactive sheet | Generate code with `ingredient_frequency` | New interactive sheet

	user_id	recipe_id	rating	rating_count
0	1533	2137	5.0	1
1	1533	8857	5.0	1
2	1533	10332	5.0	1
3	1533	10554	5.0	1
4	1533	10721	5.0	1

```
11. RECIPE ID VALIDATION
Unique recipe IDs: 231637
Unique recipe IDs in interactions: 231637
Unmatched interaction recipe IDs: 0

12. FINAL DATASET DIMENSIONS
Recipes processed: (231637, 43)
Interactions processed: (1071520, 4)
All interactions: (1132367, 5)
Zero-rated interactions: (60847, 5)

================================================================================
PHASE 3: FINAL DATA QUALITY REPORT
================================================================================

RECIPE DATASET
Total recipes: 231637
Unique recipe IDs: 231637
Duplicate recipe IDs: 0
```

```
# ============================================================
# PHASE 4.1: CONTENT-BASED RECOMMENDATION SYSTEM
# TF-IDF + COSINE SIMILARITY
# ============================================================

import pandas as pd
import numpy as np
import re
import os

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import NearestNeighbors

print("=" * 70)
print("PHASE 4.1: CONTENT-BASED RECOMMENDATION SYSTEM")
print("=" * 70)

# ------------------------------------------------
# 1. LOAD PROCESSED RECIPE DATA
# ------------------------------------------------

if "recipes_processed" not in globals():

    file_path = "/content/phase3_output/recipes_processed.csv"

    if not os.path.exists(file_path):
        raise FileNotFoundError(
            "Processed dataset not found. "
            "Upload/extract Phase3_Processed_Datasets.zip "
            "or run Phase 3 first."
        )

    recipes_processed = pd.read_csv(file_path)

# Create a separate working copy
recipe_features = recipes_processed.copy()

print("\nDataset loaded:", recipe_features.shape)

# ------------------------------------------------
# 2. HANDLE TEXT FEATURES
# ------------------------------------------------

text_columns = [
    "name",
    "ingredients_text",
    "tags_text",
    "description_clean"
]

for column in text_columns:

    if column not in recipe_features.columns:
        recipe_features[column] = ""

    recipe_features[column] = (
        recipe_features[column]
        .fillna("")
        .astype(str)
    )

# Give more importance to names and ingredients
# by including them twice in the combined text.

recipe_features["combined_features"] = (
    recipe_features["name"] + " " +
    recipe_features["name"] + " " +
    recipe_features["ingredients_text"] + " " +
    recipe_features["ingredients_text"] + " " +
    recipe_features["tags_text"] + " " +
    recipe_features["description_clean"]
)

# Normalize whitespace
recipe_features["combined_features"] = (
    recipe_features["combined_features"]
    .str.lower()
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)
```

```
# ------------------------------------------------
# 3. REMOVE EMPTY TEXT RECORDS FROM MODEL INPUT
# ------------------------------------------------

valid_text_mask = (
    recipe_features["combined_features"].str.len() > 0
)

model_recipes = (
    recipe_features.loc[valid_text_mask]
    .reset_index(drop=True)
)

print("\nRecipes available for text modelling:",
      len(model_recipes))

# ------------------------------------------------
# 4. TF-IDF VECTORIZATION
# ------------------------------------------------

tfidf = TfidfVectorizer(
    stop_words="english",
    max_features=50000,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True,
    dtype=np.float32
)

tfidf_matrix = tfidf.fit_transform(
    model_recipes["combined_features"]
)

print("\nTF-IDF VECTORIZATION COMPLETED")

print("TF-IDF matrix shape:", tfidf_matrix.shape)

print("Number of vocabulary terms:",
      len(tfidf.vocabulary_))

print("Matrix type:", type(tfidf_matrix).__name__)

print("Matrix non-zero values:", tfidf_matrix.nnz)

# ------------------------------------------------
# 5. BUILD COSINE NEAREST-NEIGHBOUR MODEL
# ------------------------------------------------

content_model = NearestNeighbors(
    metric="cosine",
    algorithm="brute",
    n_jobs=-1
)

content_model.fit(tfidf_matrix)

print("\nCOSINE NEAREST-NEIGHBOUR MODEL TRAINED!")

# ------------------------------------------------
# 6. CREATE RECIPE ID TO MATRIX INDEX MAPPING
# ------------------------------------------------

recipe_id_to_index = {
    recipe_id: index
    for index, recipe_id in enumerate(model_recipes["id"])
}

# ------------------------------------------------
# 7. RECOMMEND SIMILAR RECIPES
# ------------------------------------------------

def recommend_similar_recipes(recipe_id, top_n=10):

    if recipe_id not in recipe_id_to_index:

        print("Recipe ID not found in recommendation model.")
        return pd.DataFrame()

    recipe_index = recipe_id_to_index[recipe_id]

    # Retrieve nearest recipes
```

```
    distances, indices = content_model.kneighbors(
        tfidf_matrix[recipe_index],
        n_neighbors=min(top_n + 1, len(model_recipes))
    )

    recommendations = []

    for distance, index in zip(
        distances[0],
        indices[0]
    ):

        # Exclude the original recipe
        if index == recipe_index:
            continue

        row = model_recipes.iloc[index]

        similarity = 1 - distance

        recommendations.append({
            "recipe_id": row["id"],
            "recipe_name": row["name"],
            "similarity_score": round(
                float(similarity), 4
            ),
            "minutes": row["minutes_clean"],
            "calories": row["calories"],
            "ingredients": row["ingredients_text"]
        })

        if len(recommendations) == top_n:
            break

    return pd.DataFrame(recommendations)


# ------------------------------------------------
# 8. SEARCH RECIPES BY NAME
# ------------------------------------------------

def search_recipe(search_text, limit=10):

    search_text = str(search_text).lower().strip()

    results = model_recipes[
        model_recipes["name"]
        .str.lower()
        .str.contains(search_text, regex=False, na=False)
    ]

    return results[
        ["id", "name", "minutes_clean", "calories"]
    ].head(limit)


# ------------------------------------------------
# 9. TEST THE RECOMMENDATION ENGINE
# ------------------------------------------------

print("\n" + "=" * 70)
print("TEST 1: SEARCH FOR A RECIPE")
print("=" * 70)

search_results = search_recipe("chicken", limit=10)

display(search_results)

# Select a recipe automatically for testing
if len(search_results) > 0:

    selected_recipe_id = search_results.iloc[0]["id"]

    selected_recipe_name = search_results.iloc[0]["name"]

    print("\nSelected recipe:", selected_recipe_name)
    print("Recipe ID:", selected_recipe_id)

    print("\n" + "=" * 70)
    print("TEST 2: SIMILAR RECIPE RECOMMENDATIONS")
    print("=" * 70)
```

```
        similar_recipes = recommend_similar_recipes(
            selected_recipe_id,
            top_n=10
        )

        display(similar_recipes)

    else:

        print("No matching recipe found for testing.")

    print("\nPHASE 4.1 COMPLETED!")
```

```
======================================================================
PHASE 4.1: CONTENT-BASED RECOMMENDATION SYSTEM
======================================================================

Dataset loaded: (231637, 43)

Recipes available for text modelling: 231637

TF-IDF VECTORIZATION COMPLETED
TF-IDF matrix shape: (231637, 50000)
Number of vocabulary terms: 50000
Matrix type: csr_matrix
Matrix non-zero values: 21634158

COSINE NEAREST-NEIGHBOUR MODEL TRAINED!

======================================================================
TEST 1: SEARCH FOR A RECIPE
======================================================================
```

	id	name	minutes_clean	calories
15	63986	chicken lickin good pork chops	500.0	105.7
22	83873	crispy crunchy chicken	35.0	335.8
57	32169	make that chicken dance salsa pasta	30.0	255.2
75	42570	pick me up party chicken kabobs	30.0	213.6
85	103948	smells like sunday chicken fricassee with mea...	90.0	692.2
91	98930	steamed chicken cutlets in packages	35.0	190.5
121	42508	1 00 tangy chicken recipe	280.0	969.6
171	165856	chicken fried fish fingers	135.0	597.1
179	137434	creamy chicken curry	315.0	198.2
188	91626	evil chicken	255.0	761.1

```
Selected recipe: chicken lickin  good  pork chops
Recipe ID: 63986

======================================================================
TEST 2: SIMILAR RECIPE RECOMMENDATIONS
======================================================================
```

	recipe_id	recipe_name	similarity_score	minutes	calories	ingredients
0	35097	chicken lickin good pork chops	0.8043	380.0	361.1	lean pork chops flour salt garlic powder oil c...
1	69846	chicken fried pork chops	0.3734	370.0	465.4	flour salt dry mustard garlic powder pork chop...
2	280390	crock pot pork chops	0.3360	315.0	258.4	pork chops chicken and rice soup onion celery ...
3	86354	easy pork chops casserole	0.3203	255.0	490.3	flour salt garlic salt dry mustard lean pork c...
4	425258	slow cooker pork chops with sauerkraut	0.3142	315.0	279.4	canola oil pork chops flour salt pepper cayenn...
5	101007	crock pot gumbo pork chops	0.3074	495.0	421.3	pork chops flour salt garlic powder dry mustar...
6	79249	grannie s belly busting pork chops	0.2826	65.0	496.7	pork chops oil mustard chicken and rice soup m...
7	84363	awesome chops	0.2816	485.0	263.1	pork chops onion soup mix chicken broth
8	517313	crock pot ranch pork chops	0.2757	362.0	429.7	pork chops ranch dressing mix cream of chicken...
9	219492	pork chops garnished with peppers onions	0.2695	60.0	902.8	lean pork chops flour bell peppers yellow onio...

```
PHASE 4.1 COMPLETED!
```

Next steps: Generate code with `search_results` | New interactive sheet | Generate code with `similar_recipes` | New interactive sheet

```
# ============================================================
# PHASE 4.2: RECIPE SIMILARITY AND RECOMMENDATION REFINEMENT
# ============================================================

import pandas as pd
import numpy as np
import re
```

```
from IPython.display import display

print("=" * 70)
print("PHASE 4.2: RECIPE SIMILARITY AND RECOMMENDATION REFINEMENT")
print("=" * 70)

# ------------------------------------------------
# 1. VERIFY PHASE 4.1 VARIABLES
# ------------------------------------------------

required_variables = [
    "model_recipes",
    "tfidf_matrix",
    "content_model",
    "recipe_id_to_index"
]

missing_variables = [
    variable for variable in required_variables
    if variable not in globals()
]

if missing_variables:
    raise NameError(
        f"Missing Phase 4.1 variables: {missing_variables}. "
        "Please run Phase 4.1 first."
    )

print("\nPhase 4.1 model successfully loaded.")
print("Recipes available:", len(model_recipes))


# ------------------------------------------------
# 2. TEXT NORMALIZATION
# ------------------------------------------------

def normalize_recipe_name(name):
    """Normalize recipe names to identify duplicates."""
    name = str(name).lower()
    name = re.sub(r"[^a-z0-9\s]", " ", name)
    name = re.sub(r"\s+", " ", name)
    return name.strip()


def get_ingredient_set(text):
    """Convert ingredient text into a set of ingredients."""
    if pd.isna(text):
        return set()

    return {
        word.strip().lower()
        for word in str(text).split()
        if word.strip()
    }


# ------------------------------------------------
# 3. REFINED RECOMMENDATION FUNCTION
# ------------------------------------------------

def recommend_similar_recipes_refined(
    recipe_id,
    top_n=10,
    candidate_count=300
):

    if recipe_id not in recipe_id_to_index:
        print("Recipe ID not found in the recommendation model.")
        return pd.DataFrame()

    recipe_index = recipe_id_to_index[recipe_id]
    selected_recipe = model_recipes.iloc[recipe_index]

    selected_name = normalize_recipe_name(selected_recipe["name"])

    selected_ingredients = get_ingredient_set(
        selected_recipe["ingredients_text"]
    )

    # Retrieve nearest neighbours from the existing model
    n_candidates = min(candidate_count, len(model_recipes))
```

```
    distances, indices = content_model.kneighbors(
        tfidf_matrix[recipe_index],
        n_neighbors=n_candidates
    )

    recommendations = []
    seen_names = {selected_name}

    # Examine candidate recipes
    for distance, index in zip(distances[0], indices[0]):

        candidate = model_recipes.iloc[index]

        candidate_id = int(candidate["id"])
        candidate_name = normalize_recipe_name(candidate["name"])

        # Exclude the selected recipe
        if candidate_id == int(recipe_id):
            continue

        # Exclude recipes with duplicate names
        if candidate_name in seen_names:
            continue

        seen_names.add(candidate_name)

        candidate_ingredients = get_ingredient_set(
            candidate["ingredients_text"]
        )

        # Calculate ingredient overlap using Jaccard similarity
        union = selected_ingredients | candidate_ingredients
        intersection = selected_ingredients & candidate_ingredients

        ingredient_overlap = (
            len(intersection) / len(union)
            if union else 0
        )

        similarity = max(0, 1 - float(distance))

        recommendations.append({
            "recipe_id": candidate_id,
            "recipe_name": candidate["name"],
            "similarity_score": round(similarity, 4),
            "ingredient_overlap": round(ingredient_overlap, 4),
            "minutes": candidate.get("minutes_clean", np.nan),
            "calories": candidate.get("calories", np.nan),
            "ingredients": candidate["ingredients_text"]
        })

        if len(recommendations) == top_n:
            break

    result = pd.DataFrame(recommendations)

    return result


# ------------------------------------------------
# 4. TEST 1: RECOMMENDATIONS FOR SELECTED RECIPE
# ------------------------------------------------

test_recipe_id = 63986

selected_recipe = model_recipes[
    model_recipes["id"] == test_recipe_id
].iloc[0]

print("\n" + "=" * 70)
print("TEST 1: SELECTED RECIPE")
print("=" * 70)

print("Recipe ID:", test_recipe_id)
print("Recipe Name:", selected_recipe["name"])
print("Preparation Time:", selected_recipe["minutes_clean"])
print("Calories:", selected_recipe["calories"])

print("\n" + "=" * 70)
print("REFINED RECOMMENDATIONS")
print("=" * 70)

refined_recommendations = recommend_similar_recipes_refined(
```

```
    recipe_id=test_recipe_id,
    top_n=10
)

display(refined_recommendations)


# ------------------------------------------------
# 5. TEST 2: TEST MULTIPLE RECIPE CATEGORIES
# ------------------------------------------------

print("\n" + "=" * 70)
print("TEST 2: MULTIPLE RECIPE RECOMMENDATION TESTS")
print("=" * 70)

test_queries = ["chicken", "pasta", "vegetable", "chocolate"]

for query in test_queries:

    matching_recipes = model_recipes[
        model_recipes["name"].str.contains(
            query,
            case=False,
            na=False
        )
    ]

    if matching_recipes.empty:
        print(f"\nNo recipe found for: {query}")
        continue

    selected = matching_recipes.iloc[0]

    print("\n" + "-" * 70)
    print("SEARCH CATEGORY:", query.upper())
    print("Selected Recipe:", selected["name"])
    print("Recipe ID:", int(selected["id"]))

    recommendations = recommend_similar_recipes_refined(
        recipe_id=int(selected["id"]),
        top_n=5
    )

    display(
        recommendations[
            ["recipe_name", "similarity_score", "ingredient_overlap",
             "minutes", "calories"]
        ]
    )


# ------------------------------------------------
# 6. RECOMMENDATION QUALITY SUMMARY
# ------------------------------------------------

print("\n" + "=" * 70)
print("PHASE 4.2 VALIDATION")
print("=" * 70)

if not refined_recommendations.empty:

    print(
        "Number of recommendations:",
        len(refined_recommendations)
    )

    print(
        "Duplicate recipe names:",
        refined_recommendations["recipe_name"]
        .apply(normalize_recipe_name)
        .duplicated()
        .sum()
    )

    print(
        "Original recipe included:",
        test_recipe_id in refined_recommendations["recipe_id"].values
    )

    print(
        "Average cosine similarity:",
        round(
            refined_recommendations["similarity_score"].mean(),
```

```
            4
        )
    )

    print(
        "Average ingredient overlap:",
        round(
            refined_recommendations["ingredient_overlap"].mean(),
            4
        )
    )

print("\n" + "=" * 70)
print("PHASE 4.2 COMPLETED!")
print("=" * 70)
```

Next steps: Generate code with `refined_recommendations` New interactive sheet

```
======================================================================
PHASE 4.2: RECIPE SIMILARITY AND RECOMMENDATION REFINEMENT
======================================================================

Phase 4.1 model successfully loaded.
Recipes available: 231637

======================================================================
TEST 1: SELECTED RECIPE
======================================================================
Recipe ID: 63986
Recipe Name: chicken lickin  good  pork chops
Preparation Time: 500.0
Calories: 105.7

======================================================================
REFINED RECOMMENDATIONS
======================================================================
```

	recipe_id	recipe_name	similarity_score	ingredient_overlap	minutes	calories	ingredients
0	69846	chicken fried pork chops	0.3734	0.6471	370.0	465.4	flour salt dry mustard garlic powder pork chop...
1	280390	crock pot pork chops	0.3360	0.2941	315.0	258.4	pork chops chicken and rice soup onion celery ...
2	86354	easy pork chops casserole	0.3203	0.6875	255.0	490.3	flour salt garlic salt dry mustard lean pork c...
3	425258	slow cooker pork chops with sauerkraut	0.3142	0.3636	315.0	279.4	canola oil pork chops flour salt pepper cayenn...
4	101007	crock pot gumbo pork chops	0.3074	0.8000	495.0	421.3	pork chops flour salt garlic powder dry mustar...
5	79249	grannie s belly busting pork chops	0.2826	0.4706	65.0	496.7	pork chops oil mustard chicken and rice soup m...
6	84363	awesome chops	0.2816	0.2500	485.0	263.1	pork chops onion soup mix chicken broth
		crock pot ranch pork					pork chops ranch dressing

```
# ======================================================================
# PHASE 4.3: PERSONALIZED USER PREFERENCE MATCHING
# ======================================================================

import pandas as pd
import numpy as np
import re
from IPython.display import display

print("=" * 70)
print("PHASE 4.3: USER PREFERENCE MATCHING")
print("=" * 70)

# ------------------------------------------------
# 1. VERIFY REQUIRED DATA
# ------------------------------------------------

if "model_recipes" not in globals():
    raise NameError("Please run Phase 4.1 first.")

preference_data = model_recipes.copy()

required_columns = [
    "id", "name", "ingredients_text",
    "tags_text", "minutes_clean", "calories"
]

for col in required_columns:
    if col not in preference_data.columns:
        preference_data[col] = ""

preference_data["ingredients_text"] = (
    preference_data["ingredients_text"]
    .fillna("")
    .astype(str)
)

preference_data["tags_text"] = (
    preference_data["tags_text"]
    .fillna("")
    .astype(str)
)
```

```
preference_data["name"] = (
    preference_data["name"]
    .fillna("")
    .astype(str)
)

preference_data["minutes_clean"] = pd.to_numeric(
    preference_data["minutes_clean"],
    errors="coerce"
)

preference_data["calories"] = pd.to_numeric(
    preference_data["calories"],
    errors="coerce"
)

print("Recipes loaded:", len(preference_data))


# ------------------------------------------------
# 2. NORMALIZE USER INPUT
# ------------------------------------------------

def normalize_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()


def parse_preferences(value):
    if not value:
        return []

    return [
        normalize_text(item)
        for item in value.split(",")
        if normalize_text(item)
    ]


# ------------------------------------------------
# 3. INGREDIENT MATCHING
# ------------------------------------------------

def find_ingredient_matches(recipe_text, ingredient_list):

    recipe_text = " " + normalize_text(recipe_text) + " "

    matches = []

    for ingredient in ingredient_list:

        search_term = " " + ingredient + " "

        if search_term in recipe_text:
            matches.append(ingredient)

    return matches


# ------------------------------------------------
# 4. PERSONALIZED RECOMMENDATION FUNCTION
# ------------------------------------------------

def recommend_for_user(
    preferred_ingredients,
    disliked_ingredients=None,
    meal_type=None,
    max_minutes=None,
    top_n=10
):

    preferred = parse_preferences(preferred_ingredients)

    disliked = parse_preferences(disliked_ingredients)

    if not preferred and not disliked and not meal_type and max_minutes is None:
        print("Please provide at least one user preference.")
        return pd.DataFrame()

    results = preference_data.copy()
```

```
# Calculate preferred ingredient matches
results["preferred_matches"] = results[
    "ingredients_text"
].apply(
    lambda x: find_ingredient_matches(x, preferred)
)

# Calculate disliked ingredient matches
results["disliked_matches"] = results[
    "ingredients_text"
].apply(
    lambda x: find_ingredient_matches(x, disliked)
)

# Preferred ingredient score
if preferred:
    results["ingredient_score"] = (
        results["preferred_matches"].apply(len) / len(preferred)
    )
else:
    results["ingredient_score"] = 0.0

# Disliked ingredient penalty
if disliked:
    results["dislike_penalty"] = (
        results["disliked_matches"].apply(len) / len(disliked)
    )
else:
    results["dislike_penalty"] = 0.0

# Meal type matching
if meal_type:

    normalized_meal = normalize_text(meal_type)

    results["meal_text"] = (
        results["tags_text"] + " " + results["name"]
    ).apply(normalize_text)

    results["meal_match"] = results["meal_text"].apply(
        lambda text: (
            normalized_meal in text
            or normalized_meal.replace(" ", "-") in text
        )
    )

    results["meal_score"] = results["meal_match"].astype(float)

else:
    results["meal_score"] = 0.0

# Preparation time score
if max_minutes is not None:

    results["time_score"] = (
        results["minutes_clean"].notna()
        & (results["minutes_clean"] <= max_minutes)
    ).astype(float)

else:
    results["time_score"] = 0.0

# ------------------------------------------------
# 5. COMBINED PERSONALIZATION SCORE
# ------------------------------------------------

active_weights = {
    "ingredient_score": 0.65,
    "meal_score": 0.20,
    "time_score": 0.15
}

active_features = []

if preferred:
    active_features.append("ingredient_score")

if meal_type:
    active_features.append("meal_score")

if max_minutes is not None:
```

```
            active_features.append("time_score")

        total_weight = sum(
            active_weights[feature]
            for feature in active_features
        )

        results["preference_score"] = 0.0

        for feature in active_features:

            results["preference_score"] += (
                results[feature]
                * active_weights[feature]
                / total_weight
            )

        # Apply disliked ingredient penalty
        results["preference_score"] = (
            results["preference_score"]
            - 0.30 * results["dislike_penalty"]
        )

        results["preference_score"] = results[
            "preference_score"
        ].clip(0, 1)

        # Rank highest scoring recipes first
        results = results.sort_values(
            by="preference_score",
            ascending=False
        )

        # Remove duplicate recipe names
        results["normalized_name"] = results["name"].apply(
            normalize_text
        )

        results = results.drop_duplicates(
            subset="normalized_name"
        )

        # Return only recipes with a positive preference score
        results = results[
            results["preference_score"] > 0
        ]

        output_columns = [
            "id",
            "name",
            "preference_score",
            "preferred_matches",
            "disliked_matches",
            "minutes_clean",
            "calories"
        ]

        return results[output_columns].head(top_n).reset_index(drop=True)


    # ------------------------------------------------
    # 6. TEST USER PROFILE
    # ------------------------------------------------

    user_preferences = {
        "preferred_ingredients": "garlic, tomato, onion, olive oil",
        "disliked_ingredients": "mushrooms, cilantro",
        "meal_type": "main dish",
        "max_minutes": 60
    }

    print("\n" + "=" * 70)
    print("USER PREFERENCE PROFILE")
    print("=" * 70)

    print("Preferred ingredients:",
          user_preferences["preferred_ingredients"])

    print("Disliked ingredients:",
          user_preferences["disliked_ingredients"])

    print("Preferred meal type:",
```

```
      user_preferences["meal_type"])

print("Maximum preparation time:",
      user_preferences["max_minutes"], "minutes")


# ------------------------------------------------
# 7. GENERATE PERSONALIZED RECOMMENDATIONS
# ------------------------------------------------

print("\n" + "=" * 70)
print("PERSONALIZED RECOMMENDATIONS")
print("=" * 70)

personalized_recommendations = recommend_for_user(
    preferred_ingredients=user_preferences["preferred_ingredients"],
    disliked_ingredients=user_preferences["disliked_ingredients"],
    meal_type=user_preferences["meal_type"],
    max_minutes=user_preferences["max_minutes"],
    top_n=10
)

display(personalized_recommendations)


# ------------------------------------------------
# 8. VALIDATE RESULTS
# ------------------------------------------------

print("\n" + "=" * 70)
print("PHASE 4.3 VALIDATION")
print("=" * 70)

if not personalized_recommendations.empty:

    print(
        "Recommendations generated:",
        len(personalized_recommendations)
    )

    print(
        "Average preference score:",
        round(
            personalized_recommendations[
                "preference_score"
            ].mean(),
            4
        )
    )

    print(
        "Recipes containing preferred ingredients:",
        (personalized_recommendations[
            "preferred_matches"
        ].apply(len) > 0).sum()
    )

    print(
        "Recipes containing disliked ingredients:",
        (personalized_recommendations[
            "disliked_matches"
        ].apply(len) > 0).sum()
    )

    print(
        "Duplicate recipe names:",
        personalized_recommendations["name"]
        .apply(normalize_text)
        .duplicated()
        .sum()
    )

    print(
        "Recipes exceeding time limit:",
        (
            personalized_recommendations[
                "minutes_clean"
            ] > user_preferences["max_minutes"]
        ).sum()
    )

else:
```

```
    print("No recommendations generated. Check the user preferences.")

print("\n" + "=" * 70)
print("PHASE 4.3 COMPLETED!")
print("=" * 70)
```

```
======================================================================
PHASE 4.3: USER PREFERENCE MATCHING
======================================================================
Recipes loaded: 231637

======================================================================
USER PREFERENCE PROFILE
======================================================================
Preferred ingredients: garlic, tomato, onion, olive oil
Disliked ingredients: mushrooms, cilantro
Preferred meal type: main dish
Maximum preparation time: 60 minutes

======================================================================
PERSONALIZED RECOMMENDATIONS
======================================================================
```

	id	name	preference_score	preferred_matches	disliked_matches	minutes_clean	calories
0	188301	creamy baked ziti	1.0	[garlic, tomato, onion, olive oil]	[]	55.0	1021.0
1	9110	meat pie with phyllo dough	1.0	[garlic, tomato, onion, olive oil]	[]	30.0	396.3
2	316565	beef in pepper sauce	1.0	[garlic, tomato, onion, olive oil]	[]	55.0	408.0
3	51902	moroccan vegetable stew	1.0	[garlic, tomato, onion, olive oil]	[]	60.0	278.7
4	119386	angel hair pasta with vegetables pine nuts an...	1.0	[garlic, tomato, onion, olive oil]	[]	25.0	556.9
5	99814	chicken parmesan alla chippie	1.0	[garlic, tomato, onion, olive oil]	[]	25.0	1883.0
6	358883	chicken over pasta	1.0	[garlic, tomato, onion, olive oil]	[]	30.0	421.0
7	403798	chicken parm meatballs and spaghetti	1.0	[garlic, tomato, onion, olive oil]	[]	45.0	944.3
8	196758	tuna tomato pasta sauce	1.0	[garlic, tomato, onion, olive oil]	[]	25.0	735.2
9	456965	beef chapatis african version of tacos	1.0	[garlic, tomato, onion, olive oil]	[]	60.0	727.5

```
======================================================================
PHASE 4.3 VALIDATION
======================================================================
Recommendations generated: 10
Average preference score: 1.0
Recipes containing preferred ingredients: 10
Recipes containing disliked ingredients: 0
Duplicate recipe names: 0
Recipes exceeding time limit: 0

======================================================================
PHASE 4.3 COMPLETED!
======================================================================
```

Next steps: Generate code with `personalized_recommendations` New interactive sheet

```
# ================================================================
# PHASE 4.4: DIETARY RESTRICTION AND ALLERGY FILTERING
# ================================================================

import pandas as pd
import numpy as np
import re
from IPython.display import display

print("=" * 70)
print("PHASE 4.4: DIETARY RESTRICTION AND ALLERGY FILTERING")
print("=" * 70)

# ------------------------------------------------
# 1. VERIFY DATA
```

```
# ------------------------------------------------

if "preference_data" not in globals():
    raise NameError("Please run Phase 4.3 first.")

diet_data = preference_data.copy()

diet_data["ingredients_text"] = (
    diet_data["ingredients_text"]
    .fillna("")
    .astype(str)
)

diet_data["tags_text"] = (
    diet_data["tags_text"]
    .fillna("")
    .astype(str)
)

print("Original recipes available:", len(diet_data))


# ------------------------------------------------
# 2. DEFINE DIETARY RESTRICTION KEYWORDS
# ------------------------------------------------

dietary_restrictions = {

    "vegetarian": [
        "chicken", "beef", "pork", "lamb", "mutton",
        "turkey", "duck", "bacon", "ham", "sausage",
        "fish", "salmon", "tuna", "shrimp", "prawn",
        "crab", "lobster", "anchovy", "meat",
        "gelatin", "oyster sauce", "fish sauce"
    ],

    "vegan": [
        "chicken", "beef", "pork", "lamb", "mutton",
        "turkey", "duck", "bacon", "ham", "sausage",
        "fish", "salmon", "tuna", "shrimp", "prawn",
        "crab", "lobster", "anchovy", "meat",
        "milk", "cheese", "butter", "cream",
        "yogurt", "yoghurt", "egg", "honey",
        "gelatin", "mayonnaise", "whey",
        "casein", "fish sauce", "oyster sauce"
    ],

    "gluten_free": [
        "wheat", "barley", "rye", "semolina",
        "couscous", "bulgur", "spelt",
        "wheat flour", "bread flour",
        "all purpose flour", "pasta",
        "breadcrumbs", "soy sauce"
    ],

    "dairy_free": [
        "milk", "cheese", "butter", "cream",
        "yogurt", "yoghurt", "whey",
        "casein", "ghee", "condensed milk",
        "evaporated milk", "sour cream"
    ]
}


# ------------------------------------------------
# 3. NORMALIZE INGREDIENT TEXT
# ------------------------------------------------

def normalize_ingredient_text(text):

    text = str(text).lower()

    text = re.sub(r"[^a-z0-9\s]", " ", text)

    text = re.sub(r"\s+", " ", text)

    return " " + text.strip() + " "


def find_restricted_matches(text, restricted_list):

    normalized_text = normalize_ingredient_text(text)
```

```
    matches = []

    for ingredient in restricted_list:

        normalized_ingredient = normalize_ingredient_text(
            ingredient
        ).strip()

        pattern = r"\b" + re.escape(
            normalized_ingredient
        ) + r"\b"

        if re.search(pattern, normalized_text):
            matches.append(ingredient)

    return sorted(set(matches))


# ------------------------------------------------
# 4. CREATE USER DIETARY PROFILE
# ------------------------------------------------

user_diet_profile = {

    "diet_type": "vegetarian",

    "allergies": [
        "peanuts",
        "shellfish"
    ],

    "excluded_ingredients": [
        "mushrooms",
        "cilantro"
    ]
}

print("\n" + "=" * 70)
print("USER DIETARY PROFILE")
print("=" * 70)

print("Diet type:", user_diet_profile["diet_type"])

print("Allergies:", user_diet_profile["allergies"])

print(
    "Excluded ingredients:",
    user_diet_profile["excluded_ingredients"]
)


# ------------------------------------------------
# 5. COMBINE ALL RESTRICTIONS
# ------------------------------------------------

diet_type = user_diet_profile["diet_type"].lower()

if diet_type not in dietary_restrictions:
    raise ValueError(
        "Unsupported diet type. Choose vegetarian, vegan, "
        "gluten_free or dairy_free."
    )

diet_keywords = dietary_restrictions[diet_type]

allergy_keywords = user_diet_profile["allergies"]

custom_exclusions = user_diet_profile[
    "excluded_ingredients"
]

all_restricted_keywords = list(set(
    diet_keywords
    + allergy_keywords
    + custom_exclusions
))


# ------------------------------------------------
# 6. IDENTIFY RESTRICTED INGREDIENTS
# ------------------------------------------------
```

```
diet_data["restricted_matches"] = diet_data[
    "ingredients_text"
].apply(
    lambda text: find_restricted_matches(
        text,
        all_restricted_keywords
    )
)

diet_data["restriction_violation"] = (
    diet_data["restricted_matches"].apply(len) > 0
)


# ------------------------------------------------
# 7. FILTER UNSUITABLE RECIPES
# ------------------------------------------------

filtered_recipes = diet_data[
    diet_data["restriction_violation"] == False
].copy()

filtered_recipes.reset_index(drop=True, inplace=True)

print("\n" + "=" * 70)
print("DIETARY FILTERING RESULTS")
print("=" * 70)

print("Original recipes:", len(diet_data))

print(
    "Excluded recipes:",
    len(diet_data) - len(filtered_recipes)
)

print("Remaining recipes:", len(filtered_recipes))

print(
    "Percentage remaining:",
    round(
        len(filtered_recipes) / len(diet_data) * 100,
        2
    ),
    "%"
)


# ------------------------------------------------
# 8. SHOW EXAMPLES OF EXCLUDED RECIPES
# ------------------------------------------------

excluded_recipes = diet_data[
    diet_data["restriction_violation"] == True
].copy()

print("\n" + "=" * 70)
print("SAMPLE EXCLUDED RECIPES")
print("=" * 70)

if not excluded_recipes.empty:

    display(
        excluded_recipes[
            ["id", "name", "restricted_matches"]
        ].head(10)
    )

else:
    print("No recipes were excluded.")


# ------------------------------------------------
# 9. VALIDATE FILTERING
# ------------------------------------------------

print("\n" + "=" * 70)
print("PHASE 4.4 VALIDATION")
print("=" * 70)

remaining_violations = (
    filtered_recipes["restricted_matches"]
    .apply(len)
    .gt(0)
```

```
    .sum()
)

print("Remaining recipes:", len(filtered_recipes))

print("Remaining restriction violations:", remaining_violations)

print(
    "Duplicate recipe IDs:",
    filtered_recipes["id"].duplicated().sum()
)

print(
    "Excluded recipe count:",
    len(excluded_recipes)
)

assert remaining_violations == 0, (
    "Filtering validation failed!"
)

assert filtered_recipes["id"].is_unique, (
    "Duplicate recipe IDs detected!"
)

print("\nAll filtering validation checks passed.")

# ------------------------------------------------
# 10. SAVE FILTERED DATASET
# ------------------------------------------------

filtered_recipes.to_csv(
    "/content/Phase4_4_Filtered_Recipes.csv",
    index=False
)

print(
    "\nFiltered dataset saved:",
    "/content/Phase4_4_Filtered_Recipes.csv"
)

print("\n" + "=" * 70)
print("PHASE 4.4 COMPLETED!")
print("=" * 70)
```

```
======================================================================
PHASE 4.4: DIETARY RESTRICTION AND ALLERGY FILTERING
======================================================================
Original recipes available: 231637

======================================================================
USER DIETARY PROFILE
======================================================================
Diet type: vegetarian
Allergies: ['peanuts', 'shellfish']
Excluded ingredients: ['mushrooms', 'cilantro']

======================================================================
DIETARY FILTERING RESULTS
======================================================================
Original recipes: 231637
Excluded recipes: 94975
Remaining recipes: 136662
Percentage remaining: 59.0 %

======================================================================
SAMPLE EXCLUDED RECIPES
======================================================================
```

	id	name	restricted_matches
1	31490	a bit different breakfast pizza	[sausage]
2	112140	all in the kitchen chili	[beef]
7	67888	backyard style barbecued ribs	[cilantro, pork]
12	67547	better then bush s baked beans	[bacon, chicken]
13	107517	boat house collard greens	[ham]
15	63986	chicken lickin good pork chops	[chicken, pork]
17	23933	chinese candy	[peanuts]
18	8559	chinese chop suey	[beef, pork]
22	83873	crispy crunchy chicken	[chicken]
26	54272	fool the meat eaters chili	[beef]

```
======================================================================
PHASE 4.4 VALIDATION
======================================================================
Remaining recipes: 136662
Remaining restriction violations: 0
Duplicate recipe IDs: 0
Excluded recipe count: 94975

All filtering validation checks passed.

Filtered dataset saved: /content/Phase4_4_Filtered_Recipes.csv

======================================================================
PHASE 4.4 COMPLETED!
======================================================================
```

```
# PHASE 4.4 - FINAL VALIDATION

print("=" * 70)
print("PHASE 4.4: FINAL VALIDATION")
print("=" * 70)

# Check required variables
assert "filtered_recipes" in globals()
assert "excluded_recipes" in globals()

# Check excluded ingredients
remaining_violations = (
    filtered_recipes["restricted_matches"]
    .apply(len)
    .gt(0)
    .sum()
)

# Check duplicate IDs
duplicate_ids = filtered_recipes["id"].duplicated().sum()

# Check excluded and remaining counts
print("Original recipes:", len(diet_data))
print("Excluded recipes:", len(excluded_recipes))
print("Remaining recipes:", len(filtered_recipes))
```

```
print("Remaining recipes:", len(filtered_recipes))

print("Remaining restriction violations:", remaining_violations)
print("Duplicate recipe IDs:", duplicate_ids)

# Check that no excluded IDs remain
excluded_ids = set(excluded_recipes["id"])
remaining_ids = set(filtered_recipes["id"])

print(
    "Excluded IDs still present:",
    len(excluded_ids.intersection(remaining_ids))
)

# Validation
assert remaining_violations == 0
assert duplicate_ids == 0
assert len(excluded_ids.intersection(remaining_ids)) == 0

print("\nAll structural validation checks passed.")

print("\nSaved file:")
print("/content/Phase4_4_Filtered_Recipes.csv")

print("\nPHASE 4.4 VALIDATION COMPLETED!")
```

```
======================================================================
PHASE 4.4: FINAL VALIDATION
======================================================================
Original recipes: 231637
Excluded recipes: 94975
Remaining recipes: 136662
Remaining restriction violations: 0
Duplicate recipe IDs: 0
Excluded IDs still present: 0

All structural validation checks passed.

Saved file:
/content/Phase4_4_Filtered_Recipes.csv

PHASE 4.4 VALIDATION COMPLETED!
```

```
# ============================================================
# PHASE 4.5: NUTRITION-AWARE RECOMMENDATION SCORING
# ============================================================

import pandas as pd
import numpy as np
from IPython.display import display

print("=" * 70)
print("PHASE 4.5: NUTRITION-AWARE RECOMMENDATION SCORING")
print("=" * 70)

# ------------------------------------------------
# 1. VERIFY INPUT DATA
# ------------------------------------------------

if "filtered_recipes" not in globals():
    raise NameError("Please run Phase 4.4 first.")

nutrition_data = filtered_recipes.copy()

print("Eligible recipes loaded:", len(nutrition_data))


# ------------------------------------------------
# 2. VERIFY NUTRITION COLUMNS
# ------------------------------------------------

nutrition_columns = [
    "calories",
    "total_fat_pdv",
    "sugar_pdv",
    "sodium_pdv",
    "protein_pdv",
    "saturated_fat_pdv",
    "carbohydrates_pdv"
]

missing_columns = [
```

```
    col for col in nutrition_columns
    if col not in nutrition_data.columns
]

if missing_columns:
    raise ValueError(
        f"Missing nutrition columns: {missing_columns}"
    )

for col in nutrition_columns:
    nutrition_data[col] = pd.to_numeric(
        nutrition_data[col],
        errors="coerce"
    )

# Remove records with missing or invalid nutrition values
nutrition_data[nutrition_columns] = (
    nutrition_data[nutrition_columns].replace(
        [np.inf, -np.inf], np.nan
    )
)

nutrition_data = nutrition_data.dropna(
    subset=nutrition_columns
).copy()

print("Recipes with complete nutrition data:", len(nutrition_data))


# ------------------------------------------------
# 3. DEFINE USER NUTRITION PROFILE
# ------------------------------------------------

user_nutrition_profile = {
    "max_calories": 500,
    "prioritize_protein": True,
    "prefer_low_sugar": True,
    "prefer_low_sodium": True,
    "prefer_low_saturated_fat": True
}

print("\n" + "=" * 70)
print("USER NUTRITION PROFILE")
print("=" * 70)

for key, value in user_nutrition_profile.items():
    print(key, ":", value)


# ------------------------------------------------
# 4. APPLY CALORIE LIMIT
# ------------------------------------------------

before_calorie_filter = len(nutrition_data)

nutrition_data = nutrition_data[
    nutrition_data["calories"].between(
        0,
        user_nutrition_profile["max_calories"]
    )
].copy()

after_calorie_filter = len(nutrition_data)

print("\n" + "=" * 70)
print("CALORIE FILTERING")
print("=" * 70)

print("Before calorie filtering:", before_calorie_filter)

print("After calorie filtering:", after_calorie_filter)

print(
    "Recipes excluded:",
    before_calorie_filter - after_calorie_filter
)


# ------------------------------------------------
# 5. NORMALIZE NUTRITION VALUES
# ------------------------------------------------
```

```
# Percentile ranking maps values to a range from 0 to 1.
# Higher percentile means a higher original value.

nutrition_data["protein_score"] = (
    nutrition_data["protein_pdv"].rank(pct=True)
)

nutrition_data["sugar_score"] = (
    1 - nutrition_data["sugar_pdv"].rank(pct=True)
)

nutrition_data["sodium_score"] = (
    1 - nutrition_data["sodium_pdv"].rank(pct=True)
)

nutrition_data["saturated_fat_score"] = (
    1 - nutrition_data["saturated_fat_pdv"].rank(pct=True)
)

nutrition_data["calorie_score"] = (
    1 - nutrition_data["calories"].rank(pct=True)
)


# ------------------------------------------------
# 6. CALCULATE NUTRITION SUITABILITY SCORE
# ------------------------------------------------

weights = {
    "protein_score": 0.35,
    "sugar_score": 0.20,
    "sodium_score": 0.20,
    "saturated_fat_score": 0.15,
    "calorie_score": 0.10
}

nutrition_data["nutrition_score"] = 0.0

active_features = []

if user_nutrition_profile["prioritize_protein"]:
    active_features.append("protein_score")

if user_nutrition_profile["prefer_low_sugar"]:
    active_features.append("sugar_score")

if user_nutrition_profile["prefer_low_sodium"]:
    active_features.append("sodium_score")

if user_nutrition_profile["prefer_low_saturated_fat"]:
    active_features.append("saturated_fat_score")

# Calorie score is always active after calorie filtering.
active_features.append("calorie_score")

total_weight = sum(
    weights[feature] for feature in active_features
)

for feature in active_features:

    nutrition_data["nutrition_score"] += (
        nutrition_data[feature]
        * weights[feature]
        / total_weight
    )

nutrition_data["nutrition_score"] = (
    nutrition_data["nutrition_score"].clip(0, 1)
)


# ------------------------------------------------
# 7. RANK NUTRITIONALLY SUITABLE RECIPES
# ------------------------------------------------

nutrition_data = nutrition_data.sort_values(
    by="nutrition_score",
    ascending=False
)

nutrition_data["normalized_name"] = (
```

```
    nutrition_data["name"]
    .fillna("")
    .astype(str)
    .str.lower()
    .str.strip()
)

nutrition_recommendations = (
    nutrition_data.drop_duplicates(
        subset="normalized_name"
    )
    .head(10)
    .reset_index(drop=True)
)


# ------------------------------------------------
# 8. DISPLAY TOP 10 RECOMMENDATIONS
# ------------------------------------------------

display_columns = [
    "id",
    "name",
    "calories",
    "protein_pdv",
    "sugar_pdv",
    "sodium_pdv",
    "saturated_fat_pdv",
    "nutrition_score"
]

print("\n" + "=" * 70)
print("TOP 10 NUTRITION-AWARE RECOMMENDATIONS")
print("=" * 70)

display(nutrition_recommendations[display_columns])


# ------------------------------------------------
# 9. VALIDATION
# ------------------------------------------------

print("\n" + "=" * 70)
print("PHASE 4.5 VALIDATION")
print("=" * 70)

print(
    "Recommendations generated:",
    len(nutrition_recommendations)
)

print(
    "Average nutrition score:",
    round(
        nutrition_recommendations["nutrition_score"].mean(),
        4
    )
)

print(
    "Recipes within calorie limit:",
    (
        nutrition_recommendations["calories"]
        <= user_nutrition_profile["max_calories"]
    ).sum()
)

print(
    "Maximum calories in recommendations:",
    nutrition_recommendations["calories"].max()
)

print(
    "Minimum nutrition score:",
    nutrition_recommendations["nutrition_score"].min()
)

print(
    "Maximum nutrition score:",
    nutrition_recommendations["nutrition_score"].max()
)
```

```
print(
    "Duplicate recipe IDs:",
    nutrition_recommendations["id"].duplicated().sum()
)

# Validation checks
assert len(nutrition_recommendations) <= 10

assert (
    nutrition_recommendations["calories"]
    <= user_nutrition_profile["max_calories"]
).all()

assert nutrition_recommendations["id"].is_unique

assert nutrition_recommendations["nutrition_score"].between(
    0, 1
).all()


# ------------------------------------------------
# 10. SAVE RESULTS
# ------------------------------------------------

nutrition_recommendations.to_csv(
    "/content/Phase4_5_Nutrition_Recommendations.csv",
    index=False
)

nutrition_data.to_csv(
    "/content/Phase4_5_Nutrition_Scored_Dataset.csv",
    index=False
)

print("\nSaved files:")
print("/content/Phase4_5_Nutrition_Recommendations.csv")
print("/content/Phase4_5_Nutrition_Scored_Dataset.csv")

print("\nAll validation checks passed.")

print("\n" + "=" * 70)
print("PHASE 4.5 COMPLETED!")
print("=" * 70)
```

```
======================================================================
PHASE 4.5: NUTRITION-AWARE RECOMMENDATION SCORING
======================================================================
Eligible recipes loaded: 136662
Recipes with complete nutrition data: 136662

======================================================================
USER NUTRITION PROFILE
======================================================================
max_calories : 500
prioritize_protein : True
prefer_low_sugar : True
prefer_low_sodium : True
prefer_low_saturated_fat : True

======================================================================
CALORIE FILTERING
======================================================================
Before calorie filtering: 136662
After calorie filtering: 106449
Recipes excluded: 30213

======================================================================
TOP 10 NUTRITION-AWARE RECOMMENDATIONS
======================================================================
```

	id	name	calories	protein_pdv	sugar_pdv	sodium_pdv	saturated_fat_pdv	nutrition_score
0	186627	tofu cutlets	142.5	36.0	0.0	1.0	1.0	0.880417
1	179699	clams in roasted vegetable broth	143.8	31.0	0.0	2.0	0.0	0.873316
2	77371	easy clam dip	65.4	22.0	0.0	2.0	0.0	0.872956
3	253537	crunchy potato chip tilapia salt n vinegar	120.0	50.0	0.0	2.0	3.0	0.871730
4	307724	herb steamed chilean sea bass	94.6	33.0	0.0	2.0	2.0	0.871143
5	93049	elegant wild boar burgers for two	176.9	49.0	0.0	0.0	7.0	0.870121
6	50186	toor dal	173.6	24.0	0.0	0.0	1.0	0.869963
7	193278	beer steamed clams	93.0	23.0	0.0	2.0	0.0	0.869153
8	386000	easy cheesy vegan burgers	265.6	35.0	0.0	0.0	1.0	0.868713
9	263745	simple herb tilapia	97.8	30.0	0.0	1.0	4.0	0.867435

```
======================================================================
PHASE 4.5 VALIDATION
======================================================================
Recommendations generated: 10
Average nutrition score: 0.8715
Recipes within calorie limit: 10
Maximum calories in recommendations: 265.6
Minimum nutrition score: 0.8674348749166267
Maximum nutrition score: 0.8804173829721276
Duplicate recipe IDs: 0

Saved files:
/content/Phase4_5_Nutrition_Recommendations.csv
/content/Phase4_5_Nutrition_Scored_Dataset.csv

All validation checks passed.

======================================================================
PHASE 4.5 COMPLETED!
======================================================================
```

```
# ============================================================
# PHASE 4.6: HYBRID PERSONALIZED RECOMMENDATION ENGINE
# ============================================================

import pandas as pd
import numpy as np
import re
from sklearn.metrics.pairwise import linear_kernel

print("=" * 75)
print("PHASE 4.6: HYBRID RECOMMENDATION ENGINE")
print("=" * 75)

# ------------------------------------------------
# 1. CHECK REQUIRED VARIABLES
# ------------------------------------------------
```

```
required_variables = [
    "filtered_recipes",
    "preference_data",
    "model_recipes",
    "tfidf_matrix",
    "recipe_id_to_index",
    "user_preferences"
]

for variable in required_variables:
    if variable not in globals():
        raise NameError(
            f"{variable} is missing. Run the previous phases first."
        )

# ------------------------------------------------
# 2. PREPARE DATASETS & CALCULATE PREFERENCE SCORES
# ------------------------------------------------

hybrid_data = filtered_recipes.copy()

# Ensure recipe IDs are consistent
hybrid_data["id"] = pd.to_numeric(
    hybrid_data["id"], errors="coerce"
)

# Compute preference scores dynamically for all recipes
pref_temp = preference_data.copy()
preferred = [normalize_text(item) for item in user_preferences["preferred_ingredients"].split(",") if normalize_text(item)]
disliked = [normalize_text(item) for item in user_preferences["disliked_ingredients"].split(",") if normalize_text(item)]

pref_temp["preferred_matches"] = pref_temp["ingredients_text"].apply(lambda x: find_ingredient_matches(x, preferred))
pref_temp["disliked_matches"] = pref_temp["ingredients_text"].apply(lambda x: find_ingredient_matches(x, disliked))

if preferred:
    pref_temp["ingredient_score"] = pref_temp["preferred_matches"].apply(len) / len(preferred)
else:
    pref_temp["ingredient_score"] = 0.0

if disliked:
    pref_temp["dislike_penalty"] = pref_temp["disliked_matches"].apply(len) / len(disliked)
else:
    pref_temp["dislike_penalty"] = 0.0

if user_preferences.get("meal_type"):
    normalized_meal = normalize_text(user_preferences["meal_type"])
    pref_temp["meal_text"] = (pref_temp["tags_text"] + " " + pref_temp["name"]).apply(normalize_text)
    pref_temp["meal_match"] = pref_temp["meal_text"].apply(lambda text: normalized_meal in text or normalized_meal.replace(
    pref_temp["meal_score"] = pref_temp["meal_match"].astype(float)
else:
    pref_temp["meal_score"] = 0.0

if user_preferences.get("max_minutes") is not None:
    pref_temp["time_score"] = (pref_temp["minutes_clean"].notna() & (pref_temp["minutes_clean"] <= user_preferences["max_mi
else:
    pref_temp["time_score"] = 0.0

active_weights = {
    "ingredient_score": 0.65,
    "meal_score": 0.20,
    "time_score": 0.15
}

active_features = []
if preferred: active_features.append("ingredient_score")
if user_preferences.get("meal_type"): active_features.append("meal_score")
if user_preferences.get("max_minutes") is not None: active_features.append("time_score")

total_weight = sum(active_weights[feature] for feature in active_features)
pref_temp["preference_score"] = 0.0

for feature in active_features:
    pref_temp["preference_score"] += (pref_temp[feature] * active_weights[feature] / total_weight)

pref_temp["preference_score"] = (pref_temp["preference_score"] - 0.30 * pref_temp["dislike_penalty"]).clip(0, 1)

preference_scores = pref_temp[["id", "preference_score"]].copy()

# Nutrition scores created in Phase 4.5 (from nutrition_data or nutrition_recommendations context)
if "nutrition_data" in globals():
    nutrition_scores = nutrition_data[["id", "nutrition_score"]].copy()
else:
```

```
    nutrition_scores = pd.read_csv("/content/Phase4_5_Nutrition_Scored_Dataset.csv")[["id", "nutrition_score"]]

# Merge preference scores
hybrid_data = hybrid_data.merge(
    preference_scores,
    on="id",
    how="left"
)

# Merge nutrition scores
hybrid_data = hybrid_data.merge(
    nutrition_scores,
    on="id",
    how="left"
)

hybrid_data["preference_score"] = (
    hybrid_data["preference_score"].fillna(0)
)

hybrid_data["nutrition_score"] = (
    hybrid_data["nutrition_score"].fillna(0)
)

# ------------------------------------------------
# 3. SELECT A SEED RECIPE
# ------------------------------------------------

search_term = "vegetable"
search_results = search_recipe(search_term, limit=10)

print("\nSEED RECIPE SEARCH RESULTS")
print(search_results[["id", "name"]].to_string(index=False))

if len(search_results) == 0:
    raise ValueError("No matching seed recipe found.")

seed_recipe_id = int(search_results.iloc[0]["id"])
seed_recipe_name = search_results.iloc[0]["name"]

print("\nSelected seed recipe:", seed_recipe_name)
print("Seed recipe ID:", seed_recipe_id)

# ------------------------------------------------
# 4. CALCULATE CONTENT SIMILARITY
# ------------------------------------------------

if seed_recipe_id not in recipe_id_to_index:
    raise ValueError("Seed recipe is missing from the TF-IDF model.")

seed_index = recipe_id_to_index[seed_recipe_id]

# Find eligible recipe indexes
eligible_indices = hybrid_data["id"].map(
    recipe_id_to_index
)

valid_mask = eligible_indices.notna()
hybrid_data = hybrid_data.loc[valid_mask].copy()
hybrid_data["model_index"] = eligible_indices.loc[
    valid_mask
].astype(int)

# Calculate similarity only against eligible recipes
similarity_values = linear_kernel(
    tfidf_matrix[seed_index],
    tfidf_matrix[hybrid_data["model_index"].values]
).flatten()

hybrid_data["content_score"] = similarity_values

# Remove the seed recipe itself
hybrid_data = hybrid_data[
    hybrid_data["id"] != seed_recipe_id
].copy()

# ------------------------------------------------
# 5. NORMALIZE COMPONENT SCORES
# ------------------------------------------------

for column in [
    "content_score",
    "preference_score"
```

```
    "preference_score",
    "nutrition_score"
]:
    hybrid_data[column] = pd.to_numeric(
        hybrid_data[column], errors="coerce"
    ).fillna(0).clip(0, 1)

# ------------------------------------------------
# 6. CALCULATE HYBRID SCORE
# ------------------------------------------------

hybrid_data["hybrid_score" ] = (
    0.30 * hybrid_data["content_score"] +
    0.30 * hybrid_data["preference_score"] +
    0.40 * hybrid_data["nutrition_score"]
)

# ------------------------------------------------
# 7. RANK AND REMOVE DUPLICATE RECIPE NAMES
# ------------------------------------------------

hybrid_data["normalized_name"] = (
    hybrid_data["name"]
    .fillna("")
    .astype(str)
    .str.lower()
    .str.replace(r"[^a-z0-9 ]", "", regex=True)
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

hybrid_recommendations = (
    hybrid_data
    .sort_values("hybrid_score", ascending=False)
    .drop_duplicates(subset="normalized_name")
    .head(10)
    .copy()
)

# ------------------------------------------------
# 8. DISPLAY FINAL RECOMMENDATIONS
# ------------------------------------------------

display_columns = [
    "id",
    "name",
    "minutes_clean",
    "calories",
    "content_score",
    "preference_score",
    "nutrition_score",
    "hybrid_score"
]

display_columns = [
    column for column in display_columns
    if column in hybrid_recommendations.columns
]

print("\n" + "=" * 75)
print("TOP 10 HYBRID PERSONALIZED RECOMMENDATIONS")
print("=" * 75)

display(
    hybrid_recommendations[display_columns].reset_index(drop=True)
)

# ------------------------------------------------
# 9. VALIDATION
# ------------------------------------------------

print("\n" + "=" * 75)
print("PHASE 4.6 VALIDATION")
print("=" * 75)

print("Seed recipe:", seed_recipe_name)
print("Recommendations generated:", len(hybrid_recommendations))

print(
    "Average content score:",
    round(hybrid_recommendations["content_score"].mean(), 4)
)
```

```
print(
    "Average preference score:",
    round(hybrid_recommendations["preference_score"].mean(), 4)
)

print(
    "Average nutrition score:",
    round(hybrid_recommendations["nutrition_score"].mean(), 4)
)

print(
    "Average hybrid score:",
    round(hybrid_recommendations["hybrid_score"].mean(), 4)
)

print(
    "Duplicate recipe IDs:",
    hybrid_recommendations["id"].duplicated().sum()
)

print(
    "Seed recipe included:",
    seed_recipe_id in hybrid_recommendations["id"].values
)

print(
    "Hybrid scores within [0, 1]:",
    hybrid_recommendations["hybrid_score"].between(0, 1).all()
)

# ------------------------------------------------
# 10. EXPORT RESULTS
# ------------------------------------------------

hybrid_recommendations[display_columns].to_csv(
    "/content/Phase4_6_Hybrid_Recommendations.csv",
    index=False
)

hybrid_data.to_csv(
    "/content/Phase4_6_Hybrid_Scored_Dataset.csv",
    index=False
)

print("\nFiles saved successfully:")
print("/content/Phase4_6_Hybrid_Recommendations.csv")
print("/content/Phase4_6_Hybrid_Scored_Dataset.csv")

print("\nPHASE 4.6 COMPLETED!")
```

```
================================================================================
PHASE 4.6: HYBRID RECOMMENDATION ENGINE
================================================================================

SEED RECIPE SEARCH RESULTS
    id                                      name
 87098              homemade  vegetable soup from a can
  9999                     seva   bean   vegetable stew
 11363              denny s  vegetable beef barley soup
436082     the best  vegetarian vegan vegetable samosas
493178                     1 2 3 chicken vegetable rice
121712                        1000 island vegetable dip
246548 15 minute italian chicken   rice with vegetables
159357                    24 hour green vegetable salad
502982            25 minute tunisian vegetable couscous
390750                    2bleu s vegetable barley soup

Selected seed recipe: homemade  vegetable soup from a can
Seed recipe ID: 87098

================================================================================
TOP 10 HYBRID PERSONALIZED RECOMMENDATIONS
================================================================================
```

	id	name	minutes_clean	calories	content_score	preference_score	nutrition_score	hybrid_score
0	24641	low fat macaroni with black beans	60.0	391.0	0.039467	1.0000	0.743530	0.609252
1	141970	un chili	60.0	192.7	0.046681	1.0000	0.721760	0.602708
2	181094	grilled campfire trout	45.0	58.5	0.017356	0.8375	0.862316	0.601383
3	215177	cannellini beans with rosemary	25.0	221.4	0.060413	0.8375	0.819503	0.597175
4	286212	fakes traditional greek lentil soup	55.0	368.0	0.031996	1.0000	0.717838	0.596734
5	382092	spicy low fat veggie burgers vegan gluten f...	40.0	219.4	0.018012	1.0000	0.720162	0.593468
6	264305	tofu taco burrito filling	20.0	59.2	0.044600	1.0000	0.696188	0.591855
7	424629	greek style lentils	35.0	89.0	0.028389	1.0000	0.707606	0.591559
8	176899	easy black beans	15.0	162.1	0.068805	0.8375	0.793607	0.589334
9	97267	snappy tilapia skillet	14.0	151.6	0.073138	0.8375	0.780365	0.585337

```
================================================================================
PHASE 4.6 VALIDATION
================================================================================
Seed recipe: homemade  vegetable soup from a can
Recommendations generated: 10
Average content score: 0.0429
Average preference score: 0.935
Average nutrition score: 0.7563
Average hybrid score: 0.5959
Duplicate recipe IDs: 0
Seed recipe included: False
Hybrid scores within [0, 1]: True

Files saved successfully:
/content/Phase4_6_Hybrid_Recommendations.csv
/content/Phase4_6_Hybrid_Scored_Dataset.csv
```

```
# ============================================================
# PHASE 4.6.1: REFINED HYBRID RECOMMENDATION ENGINE
# ============================================================

import os
import re
import ast
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import linear_kernel

print("=" * 75)
print("PHASE 4.6.1: REFINED HYBRID RECOMMENDATION ENGINE")
print("=" * 75)

# ============================================================
# 1. USER PROFILE
```

```
# ================================================================

user_profile = {
    "diet_type": "vegetarian",
    "allergies": ["peanuts", "shellfish"],
    "excluded_ingredients": ["mushrooms", "cilantro"],
    "max_calories": 500,
    "preferred_ingredients": ["garlic", "tomato", "onion", "olive oil"],
    "disliked_ingredients": ["mushrooms", "cilantro"]
}

print("\nUSER PROFILE")

for key, value in user_profile.items():
    print(f"{key}: {value}")

# ================================================================
# 2. CHECK REQUIRED VARIABLES
# ================================================================

required_variables = [
    "filtered_recipes",
    "preference_data",
    "tfidf_matrix",
    "recipe_id_to_index"
]

for variable in required_variables:
    if variable not in globals():
        raise NameError(
            f"{variable} is missing. Run the previous phases first."
        )

# ================================================================
# 3. LOAD NUTRITION SCORES
# ================================================================

nutrition_path = (
    "/content/Phase4_5_Nutrition_Scored_Dataset.csv"
)

if not os.path.exists(nutrition_path):
    raise FileNotFoundError(
        "Phase 4.5 nutrition-scored dataset not found. "
        "Please run Phase 4.5 first."
    )

nutrition_df = pd.read_csv(nutrition_path)

required_nutrition_columns = [
    "id",
    "nutrition_score"
]

for column in required_nutrition_columns:
    if column not in nutrition_df.columns:
        raise ValueError(
            f"Missing column in nutrition dataset: {column}"
        )

nutrition_df["id"] = pd.to_numeric(
    nutrition_df["id"], errors="coerce"
)

nutrition_df = nutrition_df.dropna(
    subset=["id"]
).copy()

nutrition_df["id"] = nutrition_df["id"].astype(int)

nutrition_df = nutrition_df[
    ["id", "nutrition_score"]
].drop_duplicates("id")

print("\nNutrition dataset loaded:", len(nutrition_df))

# ================================================================
# 4. PREPARE RECIPE DATA
# ================================================================

refined_data = filtered_recipes.copy()
```

```
refined_data["id"] = pd.to_numeric(
    refined_data["id"], errors="coerce"
)

refined_data = refined_data.dropna(
    subset=["id"]
).copy()

refined_data["id"] = refined_data["id"].astype(int)

# Remove duplicate IDs if any
refined_data = refined_data.drop_duplicates(
    subset="id"
).copy()

print("Eligible recipes loaded:", len(refined_data))

# ============================================================
# 5. PREPARE INGREDIENT TEXT
# ============================================================

# Reuse ingredients_text from Phase 4.3 where available.
# Otherwise construct it from the ingredients column.

if "ingredients_text" in refined_data.columns:

    refined_data["ingredient_check_text"] = (
        refined_data["ingredients_text"]
        .fillna("")
        .astype(str)
        .str.lower()
    )

elif "ingredients_list" in refined_data.columns:

    def convert_ingredients(value):

        if isinstance(value, list):
            return " ".join(map(str, value)).lower()

        if isinstance(value, str):

            try:
                parsed = ast.literal_eval(value)

                if isinstance(parsed, list):
                    return " ".join(map(str, parsed)).lower()

            except (ValueError, SyntaxError):
                pass

            return value.lower()

        return ""

    refined_data["ingredient_check_text"] = (
        refined_data["ingredients_list"].apply(
            convert_ingredients
        )
    )

elif "ingredients" in refined_data.columns:

    def convert_ingredients(value):

        if isinstance(value, list):
            return " ".join(map(str, value)).lower()

        if isinstance(value, str):

            try:
                parsed = ast.literal_eval(value)

                if isinstance(parsed, list):
                    return " ".join(map(str, parsed)).lower()

            except (ValueError, SyntaxError):
                pass

            return value.lower()

        return ""
```

```
    refined_data["ingredient_check_text"] = (
        refined_data["ingredients"].apply(
            convert_ingredients
        )
    )

else:
    raise ValueError(
        "No ingredients column found in filtered_recipes."
    )

# Flag recipes without ingredient information
refined_data["ingredient_data_missing"] = (
    refined_data["ingredient_check_text"].str.strip() == ""
)

# ============================================================
# 6. DEFINE DIETARY AND ALLERGY RESTRICTIONS
# ============================================================

vegetarian_terms = [
    "chicken", "beef", "pork", "turkey", "duck",
    "lamb", "mutton", "veal", "goat meat",
    "bacon", "ham", "sausage", "pepperoni",
    "prosciutto", "lard", "suet", "gelatin",
    "fish", "salmon", "tuna", "tilapia", "cod",
    "trout", "anchovy", "sardine", "mackerel",
    "shrimp", "prawn", "crab", "lobster",
    "clam", "mussel", "oyster", "scallop",
    "crayfish", "crawfish",
    "fish sauce", "oyster sauce",
    "chicken broth", "beef broth",
    "chicken stock", "beef stock",
    "poultry stock", "fish stock"
]

peanut_terms = [
    "peanut", "peanuts",
    "groundnut", "groundnuts",
    "peanut butter", "peanut oil"
]

shellfish_terms = [
    "shrimp", "prawn", "crab", "lobster",
    "clam", "mussel", "oyster", "scallop",
    "crayfish", "crawfish", "shellfish"
]

custom_exclusion_terms = [
    "mushroom", "mushrooms",
    "cilantro", "coriander leaf",
    "fresh coriander"
]

def make_pattern(terms):

    return (
        r"(?<![a-z])(?:"
        + "|".join(re.escape(term) for term in terms)
        + r")(?![a-z])"
    )

# ============================================================
# 7. APPLY RESTRICTION FILTERS
# ============================================================

ingredient_text = refined_data[
    "ingredient_check_text"
]

refined_data["vegetarian_violation"] = (
    ingredient_text.str.contains(
        make_pattern(vegetarian_terms),
        regex=True,
        na=False
    )
)

refined_data["peanut_violation"] = (
    ingredient_text.str.contains(
        make_pattern(peanut_terms),
```

```
        regex=True,
        na=False
    )
)

refined_data["shellfish_violation"] = (
    ingredient_text.str.contains(
        make_pattern(shellfish_terms),
        regex=True,
        na=False
    )
)

refined_data["custom_violation"] = (
    ingredient_text.str.contains(
        make_pattern(custom_exclusion_terms),
        regex=True,
        na=False
    )
)

before_filtering = len(refined_data)

# Exclude recipes with missing ingredient information
# because their restrictions cannot be checked.

violation_columns = [
    "vegetarian_violation",
    "peanut_violation",
    "shellfish_violation",
    "custom_violation",
    "ingredient_data_missing"
]

refined_data = refined_data[
    ~refined_data[violation_columns].any(axis=1)
].copy()

print("\nDIETARY AND ALLERGY FILTERING")
print("Recipes before filtering:", before_filtering)
print("Recipes after filtering:", len(refined_data))
print("Recipes excluded:", before_filtering - len(refined_data))

# ================================================================
# 8. APPLY CALORIE FILTER
# ================================================================

refined_data["calories"] = pd.to_numeric(
    refined_data["calories"],
    errors="coerce"
)

before_calorie_filter = len(refined_data)

refined_data = refined_data[
    refined_data["calories"].notna() &
    (refined_data["calories"] >= 0) &
    (refined_data["calories"] <= user_profile["max_calories"])
].copy()

print("\nCALORIE FILTERING")
print("Before:", before_calorie_filter)
print("After:", len(refined_data))
print(
    "Excluded:",
    before_calorie_filter - len(refined_data)
)

# ================================================================
# 9. CALCULATE USER PREFERENCE SCORES
# ================================================================

# Calculate scores directly instead of expecting
# preference_score to already exist in preference_data.

pref_temp = preference_data.copy()

pref_temp["id"] = pd.to_numeric(
    pref_temp["id"], errors="coerce"
)

pref_temp = pref_temp.dropna(
```

```
    subset=["id"]
).copy()

pref_temp["id"] = pref_temp["id"].astype(int)

# Get ingredient text
if "ingredients_text" in pref_temp.columns:

    pref_temp["ingredients_text"] = (
        pref_temp["ingredients_text"]
        .fillna("")
        .astype(str)
        .str.lower()
    )

else:
    raise ValueError(
        "ingredients_text is missing from preference_data."
    )

preferred = user_profile["preferred_ingredients"]
disliked = user_profile["disliked_ingredients"]

def count_matches(text, terms):

    text = str(text).lower()
    matches = 0

    for term in terms:

        pattern = (
            r"(?<![a-z])" +
            re.escape(term.lower()) +
            r"(?![a-z])"
        )

        if re.search(pattern, text):
            matches += 1

    return matches

pref_temp["preferred_match_count"] = (
    pref_temp["ingredients_text"].apply(
        lambda text: count_matches(text, preferred)
    )
)

pref_temp["disliked_match_count"] = (
    pref_temp["ingredients_text"].apply(
        lambda text: count_matches(text, disliked)
    )
)

if preferred:

    pref_temp["ingredient_score"] = (
        pref_temp["preferred_match_count"] / len(preferred)
    )

else:
    pref_temp["ingredient_score"] = 0.0

if disliked:

    pref_temp["dislike_penalty"] = (
        pref_temp["disliked_match_count"] / len(disliked)
    )

else:
    pref_temp["dislike_penalty"] = 0.0

# Optional meal type score
meal_type = (
    user_preferences.get("meal_type", "")
    if "user_preferences" in globals()
    else ""
)

if meal_type and "tags_text" in pref_temp.columns:

    meal_text = (
        pref_temp["tags_text"].fillna("") + " " +
```

```
            pref_temp["name"].fillna("")
        ).str.lower()

        normalized_meal = str(meal_type).lower().replace("_", " ")

        pref_temp["meal_score"] = (
            meal_text.str.contains(
                re.escape(normalized_meal),
                regex=True,
                na=False
            )
        ).astype(float)

    else:
        pref_temp["meal_score"] = 0.0

    # Optional preparation-time score
    max_minutes = (
        user_preferences.get("max_minutes")
        if "user_preferences" in globals()
        else None
    )

    if max_minutes is not None and "minutes_clean" in pref_temp.columns:

        pref_temp["time_score"] = (
            pd.to_numeric(
                pref_temp["minutes_clean"],
                errors="coerce"
            ) <= max_minutes
        ).astype(float)

    else:
        pref_temp["time_score"] = 0.0

    # Weighted preference score
    pref_temp["preference_score"] = (
        0.65 * pref_temp["ingredient_score"] +
        0.20 * pref_temp["meal_score"] +
        0.15 * pref_temp["time_score"]
    )

    pref_temp["preference_score"] = (
        pref_temp["preference_score"] -
        0.30 * pref_temp["dislike_penalty"]
    ).clip(0, 1)

    preference_scores = pref_temp[
        ["id", "preference_score"]
    ].drop_duplicates("id")

    print("\nPREFERENCE SCORES")
    print("Recipes scored:", len(preference_scores))
    print(
        "Average preference score:",
        round(preference_scores["preference_score"].mean(), 4)
    )

    # ============================================================
    # 10. MERGE PREFERENCE AND NUTRITION SCORES
    # ============================================================

    refined_data = refined_data.merge(
        preference_scores,
        on="id",
        how="left"
    )

    refined_data = refined_data.merge(
        nutrition_df,
        on="id",
        how="left"
    )

    refined_data["preference_score"] = (
        refined_data["preference_score"].fillna(0)
    )

    refined_data["nutrition_score"] = (
        refined_data["nutrition_score"].fillna(0)
    )
```

```
# ================================================================
# 11. SELECT SEED RECIPE
# ================================================================

seed_recipe_id = 87098

if seed_recipe_id not in recipe_id_to_index:

    raise ValueError(
        "Seed recipe is not available in the TF-IDF model."
    )

seed_index = recipe_id_to_index[seed_recipe_id]

seed_row = model_recipes[
    model_recipes["id"] == seed_recipe_id
]

seed_recipe_name = (
    seed_row.iloc[0]["name"]
    if len(seed_row) > 0
    else "Selected seed recipe"
)

print("\nSEED RECIPE")
print("Recipe ID:", seed_recipe_id)
print("Recipe name:", seed_recipe_name)

# ================================================================
# 12. CALCULATE CONTENT SIMILARITY
# ================================================================

refined_data["model_index"] = refined_data["id"].map(
    recipe_id_to_index
)

refined_data = refined_data.dropna(
    subset=["model_index"]
).copy()

refined_data["model_index"] = (
    refined_data["model_index"].astype(int)
)

# Remove the seed recipe
refined_data = refined_data[
    refined_data["id"] != seed_recipe_id
].copy()

similarities = linear_kernel(
    tfidf_matrix[seed_index],
    tfidf_matrix[
        refined_data["model_index"].values
    ]
).flatten()

refined_data["content_score"] = similarities

# ================================================================
# 13. CREATE CONTENT-BASED SHORTLIST
# ================================================================

candidate_count = min(2000, len(refined_data))

refined_candidates = (
    refined_data
    .sort_values(
        "content_score",
        ascending=False
    )
    .head(candidate_count)
    .copy()
)

print("\nCONTENT SIMILARITY")
print("Eligible recipes:", len(refined_data))
print("Shortlisted recipes:", len(refined_candidates))
print(
    "Average content similarity in shortlist:",
    round(refined_candidates["content_score"].mean(), 4)
)
```

```
# ================================================================
# 14. CALCULATE HYBRID SCORE
# ================================================================

score_columns = [
    "content_score",
    "preference_score",
    "nutrition_score"
]

for column in score_columns:

    refined_candidates[column] = pd.to_numeric(
        refined_candidates[column],
        errors="coerce"
    ).fillna(0).clip(0, 1)

refined_candidates["hybrid_score"] = (
    0.30 * refined_candidates["content_score"] +
    0.30 * refined_candidates["preference_score"] +
    0.40 * refined_candidates["nutrition_score"]
)

# ================================================================
# 15. REMOVE DUPLICATE RECIPE NAMES
# ================================================================

refined_candidates["normalized_name"] = (
    refined_candidates["name"]
    .fillna("")
    .astype(str)
    .str.lower()
    .str.replace(r"[^a-z0-9 ]", "", regex=True)
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

final_recommendations = (
    refined_candidates
    .sort_values(
        "hybrid_score",
        ascending=False
    )
    .drop_duplicates(
        subset="normalized_name"
    )
    .head(10)
    .copy()
)

# ================================================================
# 16. DISPLAY FINAL RECOMMENDATIONS
# ================================================================

display_columns = [
    "id",
    "name",
    "minutes_clean",
    "calories",
    "content_score",
    "preference_score",
    "nutrition_score",
    "hybrid_score"
]

display_columns = [
    column for column in display_columns
    if column in final_recommendations.columns
]

print("\n" + "=" * 75)
print("TOP 10 REFINED HYBRID RECOMMENDATIONS")
print("=" * 75)

display(
    final_recommendations[
        display_columns
    ].reset_index(drop=True)
)

# ================================================================
# 17. VALIDATION
```

```
# ============================================================

print("\n" + "=" * 75)
print("PHASE 4.6.1 VALIDATION")
print("=" * 75)

print("Seed recipe:", seed_recipe_name)

print(
    "Recommendations generated:",
    len(final_recommendations)
)

print(
    "Average content score:",
    round(final_recommendations["content_score"].mean(), 4)
)

print(
    "Average preference score:",
    round(final_recommendations["preference_score"].mean(), 4)
)

print(
    "Average nutrition score:",
    round(final_recommendations["nutrition_score"].mean(), 4)
)

print(
    "Average hybrid score:",
    round(final_recommendations["hybrid_score"].mean(), 4)
)

print(
    "Maximum recommended calories:",
    final_recommendations["calories"].max()
)

print(
    "Duplicate recipe IDs:",
    final_recommendations["id"].duplicated().sum()
)

print(
    "Seed recipe included:",
    seed_recipe_id in final_recommendations["id"].values
)

print(
    "Vegetarian keyword violations:",
    int(final_recommendations["vegetarian_violation"].sum())
)

print(
    "Peanut keyword violations:",
    int(final_recommendations["peanut_violation"].sum())
)

print(
    "Shellfish keyword violations:",
    int(final_recommendations["shellfish_violation"].sum())
)

print(
    "Custom exclusion violations:",
    int(final_recommendations["custom_violation"].sum())
)

print(
    "Missing ingredient records:",
    int(final_recommendations["ingredient_data_missing"].sum())
)

print(
    "Hybrid scores within [0, 1]:",
    final_recommendations["hybrid_score"].between(0, 1).all()
)

# ============================================================
# 18. EXPORT RESULTS
# ============================================================
```

```
recommendation_path = (
    "/content/Phase4_6_1_Refined_Hybrid_Recommendations.csv"
)

scored_path = (
    "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
)

final_recommendations[
    display_columns
].to_csv(
    recommendation_path,
    index=False
)

refined_candidates.to_csv(
    scored_path,
    index=False
)

print("\nEXPORTED FILES")
print(recommendation_path)
print(scored_path)

print("\nPHASE 4.6.1 EXECUTION COMPLETED!")
```

```
================================================================================
PHASE 4.6.1: REFINED HYBRID RECOMMENDATION ENGINE
================================================================================

USER PROFILE
diet_type: vegetarian
```

```
# ================================================================================
# PHASE 4.7: COMPLETE RECOMMENDATION ENGINE EVALUATION
# Popularity-Based vs Content-Based vs Hybrid
# ================================================================================

import os
import ast
import warnings
import numpy as np
import pandas as pd

from sklearn.metrics.pairwise import linear_kernel
from sklearn.metrics.pairwise import cosine_similarity

warnings.filterwarnings("ignore")

print("=" * 85)
print("PHASE 4.7: RECOMMENDATION ENGINE EVALUATION")
print("=" * 85)


# ================================================================================
# 1. LOAD REQUIRED DATA
# ================================================================================

# Load processed recipes
if "model_recipes" in globals():
    evaluation_recipes = model_recipes.copy()
elif "filtered_recipes" in globals():
    evaluation_recipes = filtered_recipes.copy()
elif os.path.exists("/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"):
    evaluation_recipes = pd.read_csv(
        "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
    )
elif os.path.exists("/content/recipes_processed.csv"):
    evaluation_recipes = pd.read_csv("/content/recipes_processed.csv")
else:
    raise FileNotFoundError("Processed recipe dataset was not found.")

# Load TF-IDF model and matrix
if "tfidf_matrix" not in globals() or "recipe_id_to_index" not in globals():
    raise NameError(
        "TF-IDF model not found. Run Phase 4.1 before Phase 4.7."
    )

# Load interactions
interaction_candidates = [
    "/content/interactions_processed.csv",
    "/content/interactions_all.csv",
    "/content/phase3_output/interactions_processed.csv",
    "/content/phase3_output/interactions_all.csv"
]

evaluation_interactions = None

for file_path in interaction_candidates:
    if os.path.exists(file_path):
        evaluation_interactions = pd.read_csv(file_path)
        print("Interactions loaded from:", file_path)
        break

if evaluation_interactions is None:
    for variable_name in [
        "interactions_processed",
        "interactions_all",
        "interactions"
    ]:
        if variable_name in globals():
            candidate = globals()[variable_name]
            if isinstance(candidate, pd.DataFrame):
                evaluation_interactions = candidate.copy()
                print("Interactions loaded from runtime variable:", variable_name)
                break

if evaluation_interactions is None:
    raise FileNotFoundError(
```

```
        "Interactions dataset not found. Upload or load interactions_processed.csv."
    )

# Load nutrition scores
nutrition_candidates = [
    "/content/Phase4_5_Nutrition_Scored_Dataset.csv",
    "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
]

nutrition_evaluation = None

for file_path in nutrition_candidates:
    if os.path.exists(file_path):
        nutrition_evaluation = pd.read_csv(file_path)
        print("Nutrition scores loaded from:", file_path)
        break

if nutrition_evaluation is None and "nutrition_data" in globals():
    nutrition_evaluation = nutrition_data.copy()

if nutrition_evaluation is None:
    raise FileNotFoundError("Nutrition-scored dataset was not found.")


# ============================================================================
# 2. PREPARE RECIPE DATA
# ============================================================================

evaluation_recipes = evaluation_recipes.copy()

evaluation_recipes["id"] = pd.to_numeric(
    evaluation_recipes["id"], errors="coerce"
)

evaluation_recipes = evaluation_recipes.dropna(subset=["id"])
evaluation_recipes["id"] = evaluation_recipes["id"].astype(int)

evaluation_recipes = evaluation_recipes.drop_duplicates(
    subset=["id"]
).reset_index(drop=True)

# Keep only recipes available in the TF-IDF matrix
evaluation_recipes = evaluation_recipes[
    evaluation_recipes["id"].isin(recipe_id_to_index)
].copy()

evaluation_recipes["tfidf_index"] = evaluation_recipes["id"].map(
    recipe_id_to_index
)

evaluation_recipes = evaluation_recipes.dropna(
    subset=["tfidf_index"]
).copy()

evaluation_recipes["tfidf_index"] = (
    evaluation_recipes["tfidf_index"].astype(int)
)

recipe_ids = evaluation_recipes["id"].tolist()

print("\nRECIPE DATA")
print("Recipes available for evaluation:", len(evaluation_recipes))
print("TF-IDF matrix shape:", tfidf_matrix.shape)


# ============================================================================
# 3. PREPARE INTERACTION DATA
# ============================================================================

required_columns = ["user_id", "recipe_id", "rating"]

missing_columns = [
    column for column in required_columns
    if column not in evaluation_interactions.columns
]

if missing_columns:
    raise ValueError(
        f"Interaction dataset is missing columns: {missing_columns}"
    )

evaluation_interactions = evaluation_interactions[
    required_columns
```

```
].copy()

evaluation_interactions["recipe_id"] = pd.to_numeric(
    evaluation_interactions["recipe_id"], errors="coerce"
)

evaluation_interactions["rating"] = pd.to_numeric(
    evaluation_interactions["rating"], errors="coerce"
)

evaluation_interactions = evaluation_interactions.dropna(
    subset=required_columns
)

evaluation_interactions["recipe_id"] = (
    evaluation_interactions["recipe_id"].astype(int)
)

# Exclude zero ratings and invalid ratings
evaluation_interactions = evaluation_interactions[
    evaluation_interactions["rating"].between(1, 5)
].copy()

# Keep only recipes represented in the recommendation model
evaluation_interactions = evaluation_interactions[
    evaluation_interactions["recipe_id"].isin(recipe_ids)
].copy()

# Aggregate repeated user-recipe pairs if present
evaluation_interactions = (
    evaluation_interactions
    .groupby(["user_id", "recipe_id"], as_index=False)
    .agg(rating=("rating", "mean"))
)

print("\nINTERACTION DATA")
print("Valid explicit interactions:", len(evaluation_interactions))
print("Unique users:", evaluation_interactions["user_id"].nunique())
print("Unique recipes:", evaluation_interactions["recipe_id"].nunique())


# ============================================================================
# 4. CREATE TRAINING AND TEST DATA
# ============================================================================

# Users need at least five interactions for a meaningful holdout.
MIN_USER_INTERACTIONS = 5

# Evaluate a reproducible sample to control Colab runtime.
MAX_EVALUATION_USERS = 200

TOP_K = 10
RANDOM_STATE = 42

rng = np.random.default_rng(RANDOM_STATE)

user_interaction_counts = (
    evaluation_interactions.groupby("user_id")
    .size()
)

eligible_users = user_interaction_counts[
    user_interaction_counts >= MIN_USER_INTERACTIONS
].index.to_numpy()

if len(eligible_users) == 0:
    raise ValueError(
        "No users have enough interactions for the selected evaluation."
    )

if len(eligible_users) > MAX_EVALUATION_USERS:
    selected_users = rng.choice(
        eligible_users,
        size=MAX_EVALUATION_USERS,
        replace=False
    )
else:
    selected_users = eligible_users

train_records = []
test_records = []
```

```
for user_id in selected_users:

    user_data = evaluation_interactions[
        evaluation_interactions["user_id"] == user_id
    ].copy()

    user_data = user_data.sample(
        frac=1,
        random_state=RANDOM_STATE
    )

    test_size = max(1, int(np.ceil(len(user_data) * 0.20)))

    test_part = user_data.iloc[:test_size]
    train_part = user_data.iloc[test_size:]

    if len(train_part) < 2:
        continue

    train_records.append(train_part)
    test_records.append(test_part)

if not train_records:
    raise ValueError("Unable to create a valid train-test split.")

train_interactions = pd.concat(
    train_records, ignore_index=True
)

test_interactions = pd.concat(
    test_records, ignore_index=True
)

# Positive relevance is defined as a rating of 4 or 5.
# Ratings 1, 2, and 3 are not considered relevant.
test_relevant = test_interactions[
    test_interactions["rating"] >= 4
].copy()

print("\nTRAIN-TEST SPLIT")
print("Evaluation users:", len(train_records))
print("Training interactions:", len(train_interactions))
print("Testing interactions:", len(test_interactions))
print("Relevant test interactions (rating >= 4):", len(test_relevant))
print("Relevance threshold: rating >= 4")


# ============================================================================
# 5. BUILD POPULARITY SCORES USING TRAINING DATA ONLY
# ============================================================================

# Prevent test-data leakage by calculating popularity only from training data.
popularity_counts = (
    train_interactions[
        train_interactions["rating"] >= 4
    ]
    .groupby("recipe_id")
    .size()
)

popularity_score = pd.Series(
    0.0,
    index=recipe_ids
)

if len(popularity_counts) > 0:

    popularity_score.loc[
        popularity_counts.index.intersection(popularity_score.index)
    ] = np.log1p(
        popularity_counts.loc[
            popularity_counts.index.intersection(popularity_score.index)
        ]
    )

    max_popularity = popularity_score.max()

    if max_popularity > 0:
        popularity_score = popularity_score / max_popularity

print("\nPopularity scores calculated from training data.")
```

```
# ============================================================================
# 6. PREPARE NUTRITION SCORES
# ============================================================================

nutrition_evaluation = nutrition_evaluation.copy()

if "nutrition_score" not in nutrition_evaluation.columns:
    raise ValueError(
        "The nutrition dataset does not contain nutrition_score."
    )

nutrition_evaluation["id"] = pd.to_numeric(
    nutrition_evaluation["id"], errors="coerce"
)

nutrition_evaluation["nutrition_score"] = pd.to_numeric(
    nutrition_evaluation["nutrition_score"], errors="coerce"
)

nutrition_evaluation = nutrition_evaluation.dropna(
    subset=["id"]
)

nutrition_evaluation["id"] = (
    nutrition_evaluation["id"].astype(int)
)

nutrition_map = (
    nutrition_evaluation
    .drop_duplicates("id")
    .set_index("id")["nutrition_score"]
)

nutrition_score = pd.Series(
    recipe_ids
).map(nutrition_map)

nutrition_score.index = recipe_ids

nutrition_median = nutrition_score.median()

if pd.isna(nutrition_median):
    nutrition_median = 0.5

nutrition_score = nutrition_score.fillna(
    nutrition_median
).clip(0, 1)

print("Nutrition scores prepared.")


# ============================================================================
# 7. DEFINE EVALUATION METRICS
# ============================================================================

def precision_at_k(recommended, relevant, k=10):
    recommended = recommended[:k]
    if len(recommended) == 0:
        return 0.0
    hits = len(set(recommended) & set(relevant))
    return hits / k


def recall_at_k(recommended, relevant, k=10):
    if len(relevant) == 0:
        return np.nan
    recommended = recommended[:k]
    hits = len(set(recommended) & set(relevant))
    return hits / len(relevant)


def ndcg_at_k(recommended, relevant, k=10):
    recommended = recommended[:k]
    if len(relevant) == 0:
        return np.nan
    relevance_values = [
        1 if recipe_id in relevant else 0
        for recipe_id in recommended
    ]
    discounts = np.log2(
        np.arange(2, len(relevance_values) + 2)
    )
    dcg = np.sum(
```

```
        np.array(relevance_values) / discounts
    )
    ideal_hits = min(len(relevant), k)
    ideal_dcg = np.sum(
        1 / np.log2(np.arange(2, ideal_hits + 2))
    )
    if ideal_dcg == 0:
        return 0.0
    return dcg / ideal_dcg


# ============================================================================
# 8. GENERATE THREE TYPES OF RECOMMENDATIONS
# ============================================================================

recipe_index_array = evaluation_recipes[
    "tfidf_index"
].to_numpy()

recipe_id_array = evaluation_recipes[
    "id"
].to_numpy()

# Build a fast user-to-training-interactions lookup.
train_by_user = {
    user_id: group.copy()
    for user_id, group in train_interactions.groupby("user_id")
}

relevant_by_user = {
    user_id: set(group["recipe_id"].tolist())
    for user_id, group in test_relevant.groupby("user_id")
}

# Evaluate only users with at least one relevant test recipe.
evaluation_user_list = [
    user_id for user_id in train_by_user
    if len(relevant_by_user.get(user_id, set())) > 0
]

if len(evaluation_user_list) == 0:
    raise ValueError(
        "No evaluation users have relevant test ratings (rating >= 4)."
    )

results = []
recommendation_lists = {
    "Popularity-Based": [],
    "Content-Based": [],
    "Hybrid": []
}

print("\nGENERATING RECOMMENDATIONS...")
print("Users being evaluated:", len(evaluation_user_list))

for position, user_id in enumerate(evaluation_user_list, start=1):

    user_train = train_by_user[user_id]

    seen_recipe_ids = set(
        user_train["recipe_id"].tolist()
    )

    relevant_recipe_ids = relevant_by_user[user_id]

    # User's positively rated training recipes
    positive_train = user_train[
        user_train["rating"] >= 4
    ]

    if len(positive_train) == 0:
        positive_train = user_train[
            user_train["rating"] >= 3
        ]

    if len(positive_train) == 0:
        continue

    # Map positive recipes to TF-IDF row indices
    positive_indices = [
        recipe_id_to_index[recipe_id]
        for recipe_id in positive_train["recipe_id"]
        if recipe_id in recipe_id_to_index
```

```
    if recipe_id in recipe_id_to_index
]

if len(positive_indices) == 0:
    continue

# Build user content profile by averaging positive recipe vectors (converted to a 1D array to avoid np.matrix TypeError
user_profile_vector = np.asarray(tfidf_matrix[
    positive_indices
].mean(axis=0)).flatten()

# Similarity between user profile and all candidate recipes
content_scores_all = linear_kernel(
    user_profile_vector.reshape(1, -1),
    tfidf_matrix[recipe_index_array]
).flatten()

# Exclude recipes already seen in training
unseen_mask = ~np.isin(
    recipe_id_array,
    list(seen_recipe_ids)
)

candidate_ids = recipe_id_array[unseen_mask]
candidate_content = content_scores_all[unseen_mask]

if len(candidate_ids) == 0:
    continue

candidate_popularity = popularity_score.reindex(
    candidate_ids
).fillna(0).to_numpy()

candidate_nutrition = nutrition_score.reindex(
    candidate_ids
).fillna(nutrition_median).to_numpy()

# Normalize content similarity within each user's candidate set
content_min = candidate_content.min()
content_max = candidate_content.max()

if content_max > content_min:
    normalized_content = (
        candidate_content - content_min
    ) / (content_max - content_min)
else:
    normalized_content = np.zeros_like(candidate_content)

# Three recommendation approaches
popularity_ranking = np.argsort(
    -candidate_popularity
)[:TOP_K]

content_ranking = np.argsort(
    -candidate_content
)[:TOP_K]

# Hybrid: 50% content + 30% nutrition + 20% popularity
hybrid_scores = (
    0.50 * normalized_content
    + 0.30 * candidate_nutrition
    + 0.20 * candidate_popularity
)

hybrid_ranking = np.argsort(
    -hybrid_scores
)[:TOP_K]

recommendation_methods = {
    "Popularity-Based": candidate_ids[popularity_ranking],
    "Content-Based": candidate_ids[content_ranking],
    "Hybrid": candidate_ids[hybrid_ranking]
}

for method_name, recommendations in recommendation_methods.items():

    recommendations = recommendations.tolist()

    recommendation_lists[method_name].append(
        recommendations
    )

    results.append({
```

```
            "user_id": user_id,
            "method": method_name,
            "precision_at_10": precision_at_k(
                recommendations, relevant_recipe_ids, TOP_K
            ),
            "recall_at_10": recall_at_k(
                recommendations, relevant_recipe_ids, TOP_K
            ),
            "ndcg_at_10": ndcg_at_k(
                recommendations, relevant_recipe_ids, TOP_K
            ),
            "recommended_count": len(recommendations)
        })

    if position % 25 == 0:
        print(
            f"Processed {position}/{len(evaluation_user_list)} users"
        )


# ============================================================================
# 9. CALCULATE USER-LEVEL METRICS
# ============================================================================

user_metrics = pd.DataFrame(results)

if user_metrics.empty:
    raise ValueError("No recommendation evaluation results were generated.")

summary_metrics = (
    user_metrics
    .groupby("method")
    .agg(
        users_evaluated=("user_id", "nunique"),
        precision_at_10=("precision_at_10", "mean"),
        recall_at_10=("recall_at_10", "mean"),
        ndcg_at_10=("ndcg_at_10", "mean")
    )
    .reset_index()
)


# ============================================================================
# 10. CALCULATE CATALOG COVERAGE AND DIVERSITY
# ============================================================================

catalog_size = len(recipe_ids)

coverage_results = []

for method_name, all_recommendations in recommendation_lists.items():

    if len(all_recommendations) == 0:
        continue

    unique_recommended = set()
    diversity_values = []

    for recommendations in all_recommendations:
        unique_recommended.update(recommendations)

        if len(recommendations) >= 2:
            indices = [
                recipe_id_to_index[rid]
                for rid in recommendations
                if rid in recipe_id_to_index
            ]

            if len(indices) >= 2:
                item_matrix = tfidf_matrix[indices]
                similarity_matrix = cosine_similarity(item_matrix)
                upper_triangle = similarity_matrix[
                    np.triu_indices(len(indices), k=1)
                ]
                if len(upper_triangle) > 0:
                    diversity_values.append(
                        1 - upper_triangle.mean()
                    )

    coverage = (
        len(unique_recommended) / catalog_size
        if catalog_size else 0
    )
```

```
    average_diversity = (
        np.mean(diversity_values)
        if diversity_values else 0
    )

    coverage_results.append({
        "method": method_name,
        "unique_recipes_recommended": len(unique_recommended),
        "catalog_coverage": coverage,
        "average_intra_list_diversity": average_diversity
    })

coverage_metrics = pd.DataFrame(coverage_results)

summary_metrics = summary_metrics.merge(
    coverage_metrics,
    on="method",
    how="left"
)


# ============================================================================
# 11. DISPLAY FINAL COMPARISON
# ============================================================================

print("\n" + "=" * 85)
print("PHASE 4.7: RECOMMENDATION MODEL COMPARISON")
print("=" * 85)

print("\nMODEL PERFORMANCE")
display(summary_metrics.round(4))

print("\nUSER-LEVEL EVALUATION RESULTS")
display(user_metrics.head(15).round(4))


# ============================================================================
# 12. IDENTIFY THE BEST MODEL
# ============================================================================

best_model = summary_metrics.sort_values(
    by="ndcg_at_10",
    ascending=False
).iloc[0]

print("\nBEST MODEL BASED ON NDCG@10")
print("Model:", best_model["method"])
print("NDCG@10:", round(best_model["ndcg_at_10"], 4))
print("Precision@10:", round(best_model["precision_at_10"], 4))
print("Recall@10:", round(best_model["recall_at_10"], 4))


# ============================================================================
# 13. CREATE COMPARISON CHARTS
# ============================================================================

import matplotlib.pyplot as plt

metric_columns = [
    "precision_at_10",
    "recall_at_10",
    "ndcg_at_10"
]

for metric in metric_columns:
    plt.figure(figsize=(9, 5))
    plt.bar(
        summary_metrics["method"],
        summary_metrics[metric]
    )
    plt.title(f"Recommendation Model Comparison - {metric.upper()}")
    plt.xlabel("Recommendation Model")
    plt.ylabel(metric.upper())
    plt.xticks(rotation=15)
    plt.tight_layout()
    plt.show()

# Catalog coverage comparison
plt.figure(figsize=(9, 5))
plt.bar(
    summary_metrics["method"],
    summary_metrics["catalog_coverage"]
```

```
    summary_metrics["catalog_coverage"]
)
plt.title("Catalog Coverage Comparison")
plt.xlabel("Recommendation Model")
plt.ylabel("Catalog Coverage")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

# Diversity comparison
plt.figure(figsize=(9, 5))
plt.bar(
    summary_metrics["method"],
    summary_metrics["average_intra_list_diversity"]
)
plt.title("Recommendation Diversity Comparison")
plt.xlabel("Recommendation Model")
plt.ylabel("Average Intra-List Diversity")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()


# ============================================================================
# 14. VALIDATION
# ============================================================================

print("\n" + "=" * 85)
print("PHASE 4.7 VALIDATION")
print("=" * 85)

print("Recipes evaluated:", catalog_size)
print("Users evaluated:", user_metrics["user_id"].nunique())
print("Recommendation methods:", user_metrics["method"].nunique())
print("Top-K:", TOP_K)

print(
    "Precision values valid:",
    user_metrics["precision_at_10"].between(0, 1).all()
)
print(
    "Recall values valid:",
    user_metrics["recall_at_10"].dropna().between(0, 1).all()
)
print(
    "NDCG values valid:",
    user_metrics["ndcg_at_10"].dropna().between(0, 1).all()
)
print(
    "All three methods evaluated:",
    set(user_metrics["method"].unique()) == {
        "Popularity-Based",
        "Content-Based",
        "Hybrid"
    }
)


# ============================================================================
# 15. EXPORT EVALUATION RESULTS
# ============================================================================

summary_path = "/content/Phase4_7_Model_Comparison.csv"
user_metrics_path = "/content/Phase4_7_User_Level_Metrics.csv"
coverage_path = "/content/Phase4_7_Coverage_Diversity.csv"

summary_metrics.to_csv(summary_path, index=False)
user_metrics.to_csv(user_metrics_path, index=False)
coverage_metrics.to_csv(coverage_path, index=False)

recommendation_export = []

for method_name, user_recommendations in recommendation_lists.items():
    for user_id, recommendations in zip(
        evaluation_user_list, user_recommendations
    ):
        for rank, recipe_id in enumerate(recommendations, start=1):
            recommendation_export.append({
                "user_id": user_id,
                "method": method_name,
                "rank": rank,
                "recipe_id": recipe_id
            })
```

```
recommendation_export_df = pd.DataFrame(recommendation_export)
recommendation_export_path = "/content/Phase4_7_Evaluation_Recommendations.csv"
recommendation_export_df.to_csv(recommendation_export_path, index=False)

print("\nEXPORTED FILES")
print(summary_path)
print(user_metrics_path)
print(coverage_path)
print(recommendation_export_path)

print("\n" + "=" * 85)
print("PHASE 4.7 EXECUTION COMPLETED!")
print("=" * 85)
```

```
================================================================================
PHASE 4.7: RECOMMENDATION ENGINE EVALUATION
================================================================================
Interactions loaded from: /content/phase3_output/interactions_processed.csv
Nutrition scores loaded from: /content/Phase4_5_Nutrition_Scored_Dataset.csv

RECIPE DATA
Recipes available for evaluation: 231637
TF-IDF matrix shape: (231637, 50000)

INTERACTION DATA
Valid explicit interactions: 1071520
Unique users: 196098
Unique recipes: 226590

TRAIN-TEST SPLIT
Evaluation users: 200
Training interactions: 6784
Testing interactions: 1799
Relevant test interactions (rating >= 4): 1696
Relevance threshold: rating >= 4

Popularity scores calculated from training data.
Nutrition scores prepared.

GENERATING RECOMMENDATIONS...
Users being evaluated: 197
Processed 25/197 users
Processed 50/197 users
Processed 75/197 users
Processed 100/197 users
Processed 125/197 users
Processed 150/197 users
Processed 175/197 users

================================================================================
PHASE 4.7: RECOMMENDATION MODEL COMPARISON
================================================================================

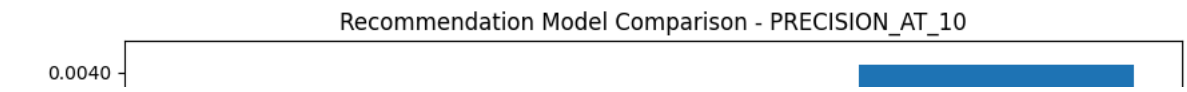
MODEL PERFORMANCE
```

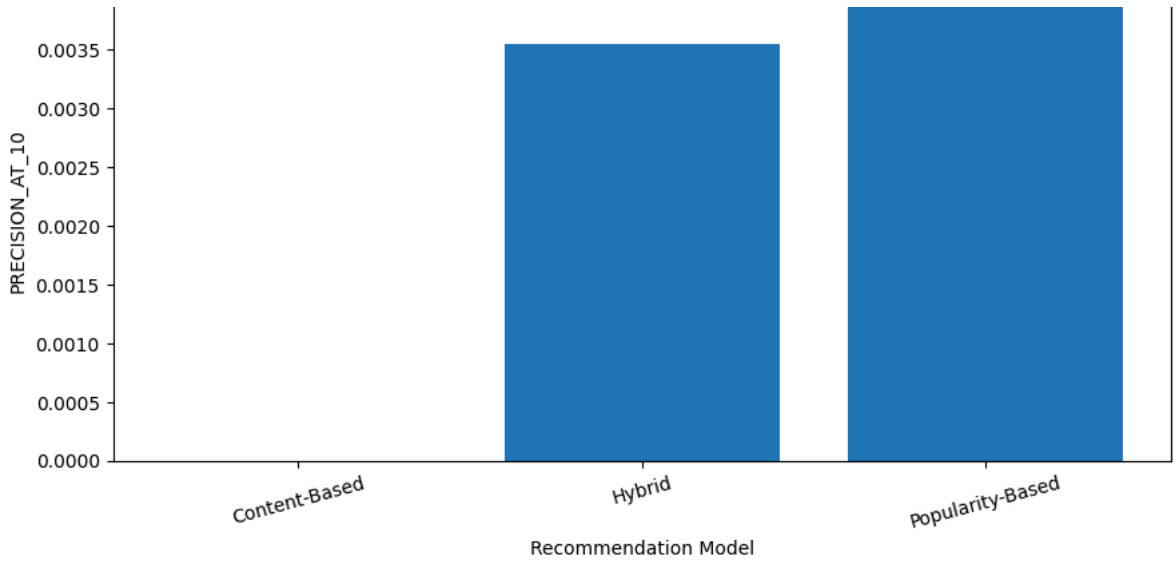
	method	users_evaluated	precision_at_10	recall_at_10	ndcg_at_10	unique_recipes_recommended	catalog_coverage	aver
0	Content-Based	197	0.0000	0.0000	0.0000	1689	0.0073	
1	Hybrid	197	0.0036	0.0051	0.0067	1353	0.0058	
2	Popularity-Based	197	0.0041	0.0120	0.0063	18	0.0001	

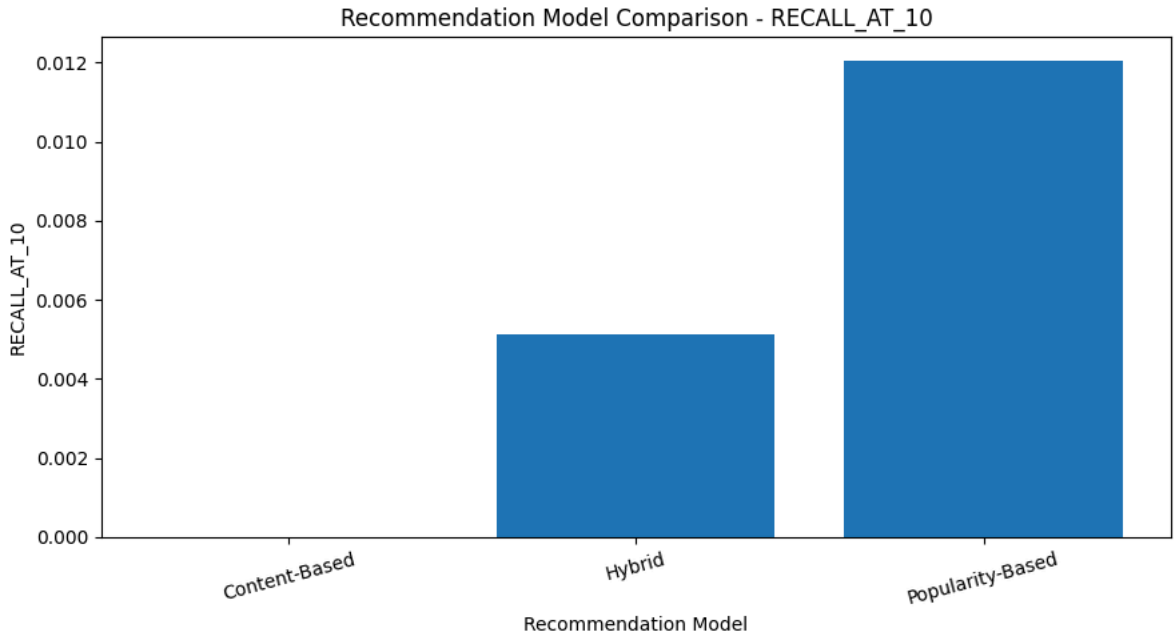
```
USER-LEVEL EVALUATION RESULTS
```

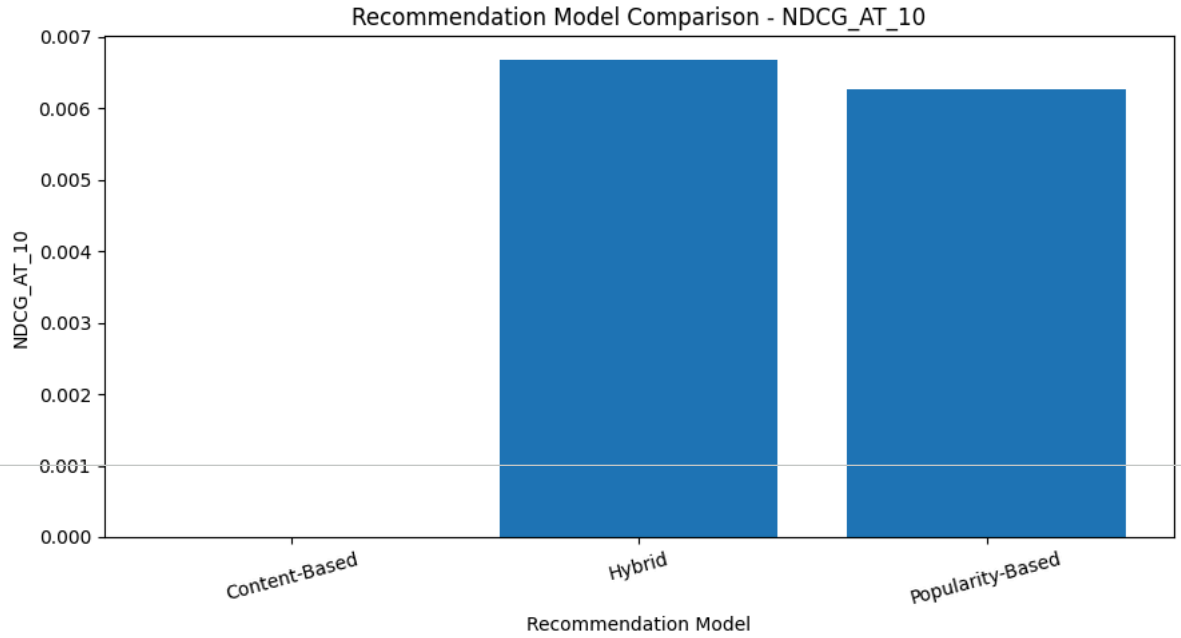
	user_id	method	precision_at_10	recall_at_10	ndcg_at_10	recommended_count
0	12491	Popularity-Based	0.0	0.0	0.0	10
1	12491	Content-Based	0.0	0.0	0.0	10
2	12491	Hybrid	0.0	0.0	0.0	10
3	26584	Popularity-Based	0.0	0.0	0.0	10
4	26584	Content-Based	0.0	0.0	0.0	10
5	26584	Hybrid	0.0	0.0	0.0	10
6	30651	Popularity-Based	0.0	0.0	0.0	10
7	30651	Content-Based	0.0	0.0	0.0	10
8	30651	Hybrid	0.0	0.0	0.0	10
9	33123	Popularity-Based	0.0	0.0	0.0	10
10	33123	Content-Based	0.0	0.0	0.0	10
11	33123	Hybrid	0.0	0.0	0.0	10
12	35267	Popularity-Based	0.0	0.0	0.0	10
13	35267	Content-Based	0.0	0.0	0.0	10
14	35267	Hybrid	0.0	0.0	0.0	10

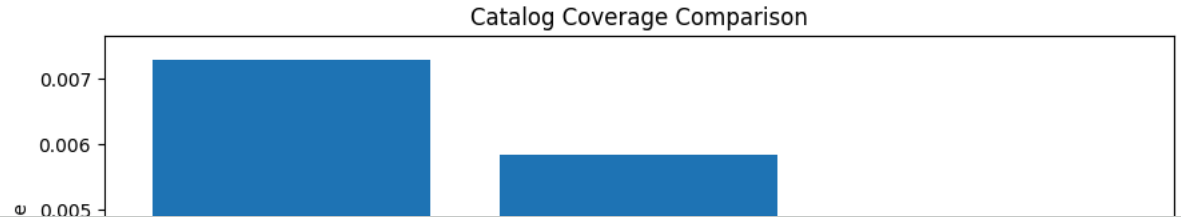
```
BEST MODEL BASED ON NDCG@10
Model: Hybrid
NDCG@10: 0.0067
Precision@10: 0.0036
Recall@10: 0.0051
```




PRECISION_AT_10
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
Content-Based
Hybrid
Popularity-Based
Recommendation Model


Recommendation Model Comparison - RECALL_AT_10
RECALL_AT_10
0.012
0.010
0.008
0.006
0.004
0.002
0.000
Content-Based
Hybrid
Popularity-Based
Recommendation Model


Recommendation Model Comparison - NDCG_AT_10
NDCG_AT_10
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
Content-Based
Hybrid
Popularity-Based
Recommendation Model


Catalog Coverage Comparison
0.007
0.006
0.005

```
# ============================================================================
# PHASE 4.7.1: IMPROVED RECOMMENDATION ENGINE EVALUATION
# Popularity-Based vs Content-Based vs Hybrid
# ============================================================================

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics.pairwise import linear_kernel, cosine_similarity

warnings.filterwarnings("ignore")

print("=" * 85)
print("PHASE 4.7.1: IMPROVED RECOMMENDATION EVALUATION")
print("=" * 85)


# ============================================================================
# 1. LOAD AND PREPARE DATA
# ============================================================================

# Recipes
if "model_recipes" in globals():
    eval_recipes = model_recipes.copy()

elif "filtered_recipes" in globals():
    eval_recipes = filtered_recipes.copy()

elif os.path.exists(
    "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
):
    eval_recipes = pd.read_csv(
        "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
    )

elif os.path.exists("/content/recipes_processed.csv"):
    eval_recipes = pd.read_csv(
        "/content/recipes_processed.csv"
    )

else:
    raise FileNotFoundError("Recipe dataset not found.")


# TF-IDF requirements
if "tfidf_matrix" not in globals():
    raise NameError("Run Phase 4.1 first: tfidf_matrix is missing.")

if "recipe_id_to_index" not in globals():
    raise NameError("Run Phase 4.1 first: recipe_id_to_index is missing.")


# Load interactions
interaction_paths = [
    "/content/phase3_output/interactions_processed.csv",
    "/content/interactions_processed.csv",
    "/content/interactions_all.csv"
]

eval_interactions = None

for file_path in interaction_paths:
    if os.path.exists(file_path):
        eval_interactions = pd.read_csv(file_path)
        print("Interactions loaded:", file_path)
        break

if eval_interactions is None:
    for variable_name in [
        "interactions_processed",
        "interactions_all",
        "interactions"
    ]:
        if variable_name in globals():
            candidate = globals()[variable_name]

            if isinstance(candidate, pd.DataFrame):
                eval_interactions = candidate.copy()
                print("Interactions loaded from:", variable_name)
```

```
                break

if eval_interactions is None:
    raise FileNotFoundError("Processed interactions not found.")


# Load nutrition scores
nutrition_paths = [
    "/content/Phase4_5_Nutrition_Scored_Dataset.csv",
    "/content/Phase4_6_1_Refined_Hybrid_Scored_Dataset.csv"
]

eval_nutrition = None

for file_path in nutrition_paths:
    if os.path.exists(file_path):
        eval_nutrition = pd.read_csv(file_path)
        print("Nutrition scores loaded:", file_path)
        break

if eval_nutrition is None and "nutrition_data" in globals():
    eval_nutrition = nutrition_data.copy()

if eval_nutrition is None:
    raise FileNotFoundError("Nutrition-scored dataset not found.")


# ============================================================================
# 2. CLEAN RECIPE DATA
# ============================================================================

eval_recipes["id"] = pd.to_numeric(
    eval_recipes["id"], errors="coerce"
)

eval_recipes = eval_recipes.dropna(
    subset=["id"]
).copy()

eval_recipes["id"] = eval_recipes["id"].astype(int)

eval_recipes = eval_recipes.drop_duplicates(
    subset=["id"]
).copy()

# Only recipes represented in the TF-IDF matrix
eval_recipes = eval_recipes[
    eval_recipes["id"].isin(recipe_id_to_index)
].copy()

eval_recipes["tfidf_index"] = eval_recipes["id"].map(
    recipe_id_to_index
).astype(int)

eval_recipes = eval_recipes.reset_index(drop=True)

recipe_ids = eval_recipes["id"].to_numpy()
recipe_indices = eval_recipes["tfidf_index"].to_numpy()

print("\nRECIPE DATA")
print("Available recipes:", len(eval_recipes))
print("TF-IDF matrix:", tfidf_matrix.shape)


# ============================================================================
# 3. CLEAN INTERACTION DATA
# ============================================================================

required_columns = ["user_id", "recipe_id", "rating"]

if not set(required_columns).issubset(eval_interactions.columns):
    raise ValueError(
        "Interactions must contain user_id, recipe_id and rating."
    )

eval_interactions = eval_interactions[
    required_columns
].copy()

eval_interactions["recipe_id"] = pd.to_numeric(
    eval_interactions["recipe_id"], errors="coerce"
)
```

```
eval_interactions["rating"] = pd.to_numeric(
    eval_interactions["rating"], errors="coerce"
)

eval_interactions = eval_interactions.dropna(
    subset=required_columns
)

eval_interactions["recipe_id"] = (
    eval_interactions["recipe_id"].astype(int)
)

# Explicit ratings only
eval_interactions = eval_interactions[
    eval_interactions["rating"].between(1, 5)
].copy()

# Remove recipes unavailable in the model
eval_interactions = eval_interactions[
    eval_interactions["recipe_id"].isin(recipe_ids)
].copy()

# Aggregate repeated user-recipe pairs
eval_interactions = (
    eval_interactions
    .groupby(["user_id", "recipe_id"], as_index=False)
    .agg(rating=("rating", "mean"))
)

print("\nINTERACTION DATA")
print("Explicit interactions:", len(eval_interactions))
print("Unique users:", eval_interactions["user_id"].nunique())
print("Unique recipes:", eval_interactions["recipe_id"].nunique())


# ============================================================================
# 4. CREATE USER-LEVEL TRAIN-TEST SPLIT
# ============================================================================

RANDOM_STATE = 42
TOP_K = 10
MAX_EVALUATION_USERS = 200
MIN_INTERACTIONS = 5

rng = np.random.default_rng(RANDOM_STATE)

user_counts = eval_interactions.groupby("user_id").size()

eligible_users = user_counts[
    user_counts >= MIN_INTERACTIONS
].index.to_numpy()

if len(eligible_users) == 0:
    raise ValueError("No eligible users found.")

if len(eligible_users) > MAX_EVALUATION_USERS:
    selected_users = rng.choice(
        eligible_users,
        size=MAX_EVALUATION_USERS,
        replace=False
    )
else:
    selected_users = eligible_users

selected_users = set(selected_users.tolist())

train_parts = []
test_parts = []

# Interactions from users not selected for evaluation remain training data.
other_users_train = eval_interactions[
    ~eval_interactions["user_id"].isin(selected_users)
].copy()

train_parts.append(other_users_train)

# Split only the selected evaluation users.
for user_id in selected_users:

    user_data = eval_interactions[
        eval_interactions["user_id"] == user_id
    ].sample(
        frac=1
```

```
        frac=1,
        random_state=RANDOM_STATE
    )

    test_size = max(
        1,
        int(np.ceil(len(user_data) * 0.20))
    )

    test_part = user_data.iloc[:test_size].copy()
    train_part = user_data.iloc[test_size:].copy()

    train_parts.append(train_part)
    test_parts.append(test_part)

train_interactions = pd.concat(
    train_parts, ignore_index=True
)

test_interactions = pd.concat(
    test_parts, ignore_index=True
)

# Relevant recipes are held-out recipes rated 4 or 5.
test_relevant = test_interactions[
    test_interactions["rating"] >= 4
].copy()

print("\nTRAIN-TEST SPLIT")
print("Evaluation users:", len(selected_users))
print("Training interactions:", len(train_interactions))
print("Testing interactions:", len(test_interactions))
print("Relevant held-out interactions:", len(test_relevant))

print(
    "Training users:",
    train_interactions["user_id"].nunique()
)


# ============================================================================
# 5. BUILD FULL-POPULATION TRAINING POPULARITY
# ============================================================================

# IMPORTANT:
# Popularity is calculated using training interactions from the full
# population, not just the selected evaluation users.

positive_train = train_interactions[
    train_interactions["rating"] >= 4
]

popularity_counts = (
    positive_train.groupby("recipe_id")
    .size()
)

popularity_score = pd.Series(
    0.0,
    index=recipe_ids
)

valid_popularity_ids = popularity_counts.index.intersection(
    popularity_score.index
)

popularity_score.loc[valid_popularity_ids] = np.log1p(
    popularity_counts.loc[valid_popularity_ids]
)

if popularity_score.max() > 0:
    popularity_score = (
        popularity_score / popularity_score.max()
    )

print("\nPOPULARITY MODEL")
print("Recipes with positive training ratings:", len(popularity_counts))
print("Unique recipes with popularity scores:", (popularity_score > 0).sum())


# ============================================================================
# 6. PREPARE NUTRITION SCORES
# ============================================================================
```